

8

Advanced Counting Techniques

- 8.1 Applications of Recurrence Relations
- 8.2 Solving Linear Recurrence Relations
- 8.3 Divide-and-Conquer Algorithms and Recurrence Relations
- 8.4 Generating Functions
- 8.5 Inclusion–Exclusion
- 8.6 Applications of Inclusion–Exclusion

Many counting problems cannot be solved easily using the methods discussed in Chapter 6. One such problem is: How many bit strings of length n do not contain two consecutive zeros? To solve this problem, let a_n be the number of such strings of length n . An argument can be given that shows that the sequence $\{a_n\}$ satisfies the recurrence relation $a_{n+1} = a_n + a_{n-1}$ and the initial conditions $a_1 = 2$ and $a_2 = 3$. This recurrence relation and the initial conditions determine the sequence $\{a_n\}$. Moreover, an explicit formula can be found for a_n from the equation relating the terms of the sequence. As we will see, a similar technique can be used to solve many different types of counting problems.

We will discuss two ways that recurrence relations play important roles in the study of algorithms. First, we will introduce an important algorithmic paradigm known as dynamic programming. Algorithms that follow this paradigm break down a problem into overlapping subproblems. The solution to the problem is then found from the solutions to the subproblems through the use of a recurrence relation. Second, we will study another important algorithmic paradigm, divide-and-conquer. Algorithms that follow this paradigm can be used to solve a problem by recursively breaking it into a fixed number of nonoverlapping subproblems until these problems can be solved directly. The complexity of such algorithms can be analyzed using a special type of recurrence relation. In this chapter we will discuss a variety of divide-and-conquer algorithms and analyze their complexity using recurrence relations.

We will also see that many counting problems can be solved using formal power series, called generating functions, where the coefficients of powers of x represent terms of the sequence we are interested in. Besides solving counting problems, we will also be able to use generating functions to solve recurrence relations and to prove combinatorial identities.

Many other kinds of counting problems cannot be solved using the techniques discussed in Chapter 6, such as: How many ways are there to assign seven jobs to three employees so that each employee is assigned at least one job? How many primes are there less than 1000? Both of these problems can be solved by counting the number of elements in the union of sets. We will develop a technique, called the principle of inclusion–exclusion, that counts the number of elements in a union of sets, and we will show how this principle can be used to solve counting problems.

The techniques studied in this chapter, together with the basic techniques of Chapter 6, can be used to solve many counting problems.

8.1 Applications of Recurrence Relations

8.1.1 Introduction

Recall from Chapter 2 that a recursive definition of a sequence specifies one or more initial terms and a rule for determining subsequent terms from those that precede them. Also, recall that a rule of the latter sort (whether or not it is part of a recursive definition) is called a **recurrence relation** and that a sequence is called a *solution* of a recurrence relation if its terms satisfy the recurrence relation.

In this section we will show that such relations can be used to study and to solve counting problems. For example, suppose that the number of bacteria in a colony doubles every hour. If a colony begins with five bacteria, how many will be present in n hours? To solve this problem, let a_n be the number of bacteria at the end of n hours. Because the number of bacteria doubles

every hour, the relationship $a_n = 2a_{n-1}$ holds whenever n is a positive integer. This recurrence relation, together with the initial condition $a_0 = 5$, uniquely determines a_n for all nonnegative integers n . We can find a formula for a_n using the iterative approach followed in Chapter 2, namely that $a_n = 5 \cdot 2^n$ for all nonnegative integers n .

Some of the counting problems that cannot be solved using the techniques discussed in Chapter 6 can be solved by finding recurrence relations involving the terms of a sequence, as was done in the problem involving bacteria. In this section we will study a variety of counting problems that can be modeled using recurrence relations. In Chapter 2 we developed methods for solving certain recurrence relation. In Section 8.2 we will study methods for finding explicit formulae for the terms of sequences that satisfy certain types of recurrence relations.

We conclude this section by introducing the algorithmic paradigm of dynamic programming. After explaining how this paradigm works, we will illustrate its use with an example.

8.1.2 Modeling With Recurrence Relations

Assessment We can use recurrence relations to model a wide variety of problems, such as finding compound interest (see Example 11 in Section 2.4), counting rabbits on an island, determining the number of moves in the Tower of Hanoi puzzle, and counting bit strings with certain properties.

Extra Examples Example 1 shows how the population of rabbits on an island can be modeled using a recurrence relation.

EXAMPLE 1 Rabbits and the Fibonacci Numbers Consider this problem, which was originally posed by Leonardo Pisano, also known as Fibonacci, in the thirteenth century in his book *Liber abaci*. A young pair of rabbits (one of each sex) is placed on an island. A pair of rabbits does not breed until they are 2 months old. After they are 2 months old, each pair of rabbits produces another pair each month, as shown in Figure 1. Find a recurrence relation for the number of pairs of rabbits on the island after n months, assuming that no rabbits ever die.

Reproducing pairs (at least two months old)	Young pairs (less than two months old)	Month	Reproducing pairs	Young pairs	Total pairs
		1	0	1	1
		2	0	1	1
		3	1	1	2
		4	1	2	3
		5	2	3	5
	 	6	3	5	8

FIGURE 1 Rabbits on an island.

The Fibonacci numbers appear in many other places in nature, including the number of petals on flowers and the number of spirals on seedheads.

Solution: Denote by f_n the number of pairs of rabbits after n months. We will show that f_n , $n = 1, 2, 3, \dots$, are the terms of the Fibonacci sequence.

The rabbit population can be modeled using a recurrence relation. At the end of the first month, the number of pairs of rabbits on the island is $f_1 = 1$. Because this pair does not breed during the second month, $f_2 = 1$ also. To find the number of pairs after n months, add the number on the island the previous month, f_{n-1} , and the number of newborn pairs, which equals f_{n-2} , because each newborn pair comes from a pair at least 2 months old.

Consequently, the sequence $\{f_n\}$ satisfies the recurrence relation

$$f_n = f_{n-1} + f_{n-2}$$

for $n \geq 3$ together with the initial conditions $f_1 = 1$ and $f_2 = 1$. Because this recurrence relation and the initial conditions uniquely determine this sequence, the number of pairs of rabbits on the island after n months is given by the n th Fibonacci number. ▶

Demo

Example 2 involves a famous puzzle.

EXAMPLE 2

Links

The Tower of Hanoi Puzzle A popular puzzle of the late nineteenth century invented by the French mathematician Édouard Lucas, called the Tower of Hanoi, consists of three pegs mounted on a board together with disks of different sizes. Initially these disks are placed on the first peg in order of size, with the largest on the bottom (as shown in Figure 2). The rules of the puzzle allow disks to be moved one at a time from one peg to another as long as a disk is never placed on top of a smaller disk. The goal of the puzzle is to have all the disks on the second peg in order of size, with the largest on the bottom.

Let H_n denote the number of moves needed to solve the Tower of Hanoi puzzle with n disks. Set up a recurrence relation for the sequence $\{H_n\}$.

Solution: Begin with n disks on peg 1. We can transfer the top $n - 1$ disks, following the rules of the puzzle, to peg 3 using H_{n-1} moves (see Figure 3 for an illustration of the pegs and disks at this point). We keep the largest disk fixed during these moves. Then, we use one move to transfer the largest disk to the second peg. Finally, we transfer the $n - 1$ disks on peg 3 to peg 2 using H_{n-1} moves, placing them on top of the largest disk, which always stays fixed on the bottom of peg 2. This shows that we can solve the Tower of Hanoi puzzle for n disks using $2H_{n-1} + 1$ moves.

We now show that we cannot solve the puzzle for n disks using fewer than $2H_{n-1} + 1$ moves. Note that when we move the largest disk, we must have already moved the $n - 1$ smaller disks onto a peg other than peg 1. Doing so requires at least H_{n-1} moves. Another move is needed to

Schemes for efficiently backing up computer files on multiple tapes or other media are based on the moves used to solve the Tower of Hanoi puzzle.

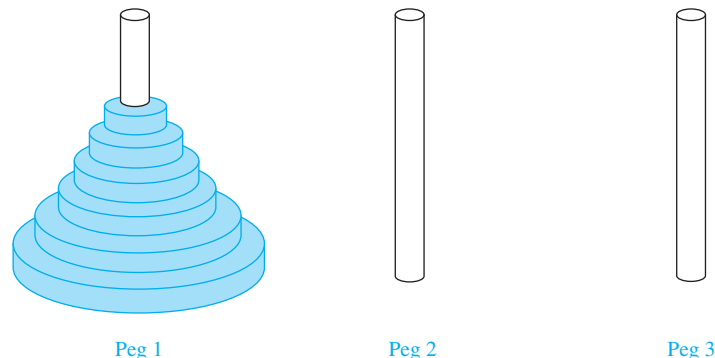


FIGURE 2 The initial position in the Tower of Hanoi.

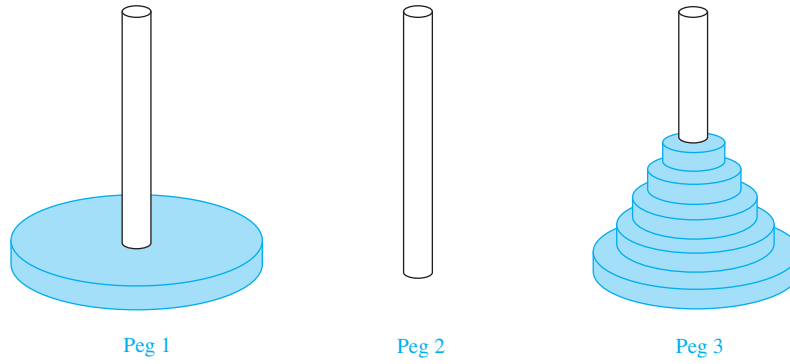


FIGURE 3 An intermediate position in the Tower of Hanoi.

transfer the largest disk. Finally, at least H_{n-1} more moves are needed to put the $n - 1$ smallest disks back on top of the largest disk. Adding the number of moves required gives us the desired lower bound.

We conclude that

$$H_n = 2H_{n-1} + 1.$$

The initial condition is $H_1 = 1$, because one disk can be transferred from peg 1 to peg 2, according to the rules of the puzzle, in one move.

We can use an iterative approach to solve this recurrence relation. Note that

$$\begin{aligned} H_n &= 2H_{n-1} + 1 \\ &= 2(2H_{n-2} + 1) + 1 = 2^2H_{n-2} + 2 + 1 \\ &= 2^2(2H_{n-3} + 1) + 2 + 1 = 2^3H_{n-3} + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}H_1 + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + \cdots + 2 + 1 \\ &= 2^n - 1. \end{aligned}$$

We have used the recurrence relation repeatedly to express H_n in terms of previous terms of the sequence. In the next to last equality, the initial condition $H_1 = 1$ has been used. The last equality is based on the formula for the sum of the terms of a geometric series, which can be found in Theorem 1 in Section 2.4.

The iterative approach has produced the solution to the recurrence relation $H_n = 2H_{n-1} + 1$ with the initial condition $H_1 = 1$. This formula can be proved using mathematical induction. This is left for the reader as Exercise 1.

A myth created to accompany the puzzle tells of a tower in Hanoi where monks are transferring 64 gold disks from one peg to another, according to the rules of the puzzle. The myth says that the world will end when they finish the puzzle. How long after the monks started will the world end if the monks take one second to move a disk?

From the explicit formula, the monks require

$$2^{64} - 1 = 18,446,744,073,709,551,615$$

moves to transfer the disks. Making one move per second, it will take them more than 500 billion years to complete the transfer, so the world should survive a while longer than it already has. ◀

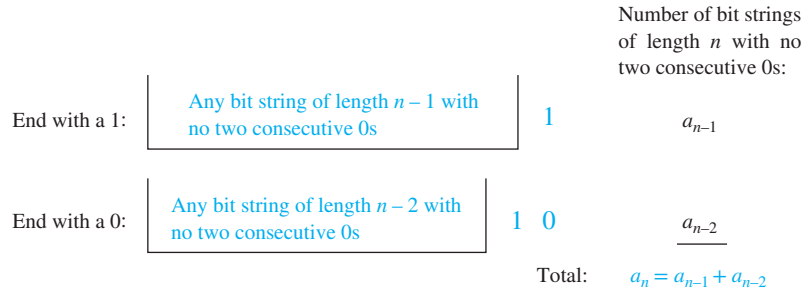


FIGURE 4 Counting bit strings of length n with no two consecutive 0s.

Links

Remark: Many people have studied variations of the original Tower of Hanoi puzzle discussed in Example 2. Some variations use more pegs, some allow disks to be of the same size, and some restrict the types of allowable disk moves. One of the oldest and most interesting variations is the **Reve's puzzle**,* proposed in 1907 by Henry Dudeney in his book *The Canterbury Puzzles*. The Reve's puzzle involves pilgrims challenged by the Reve to move a stack of cheese wheels of varying sizes from the first of four stools to another stool without ever placing a cheese wheel on one of smaller diameter. The Reve's puzzle, expressed in terms of pegs and disks, follows the same rules as the Tower of Hanoi puzzle, except that four pegs are used. Similarly, we can generalize the Tower of Hanoi puzzle where there are p pegs, where p is an integer greater than three. You may find it surprising that no one has been able to establish the minimum number of moves required to solve the generalization of this puzzle for p pegs. (Note that there have been some published claims that this problem has been solved, but these are not accepted by experts.) However, in 2014 Thierry Bousch showed that the minimum number of moves required when there are four pegs equals the number of moves used by an algorithm invented by Frame and Stewart in 1939. (See Exercises 38–45 and [St94] and [Bo14] for more information.)

Example 3 illustrates how recurrence relations can be used to count bit strings of a specified length that have a certain property.

EXAMPLE 3 Find a recurrence relation and give initial conditions for the number of bit strings of length n that do not have two consecutive 0s. How many such bit strings are there of length five?

Solution: Let a_n denote the number of bit strings of length n that do not have two consecutive 0s. We assume that $n \geq 3$, so that the bit string has at least three bits. Strings of this sort of length n can be divided into those that end in 1 and those that end in 0. The bit strings of length n ending with 1 that do not have two consecutive 0s are precisely the bit strings of length $n - 1$ with no two consecutive 0s with a 1 added at the end. Consequently, there are a_{n-1} such bit strings.

Bit strings of length n ending with a 0 that do not have two consecutive 0s must have 1 as their $(n - 1)$ st bit; otherwise they would end with a pair of 0s. Hence, the bit strings of length n ending with a 0 that have no two consecutive 0s are precisely the bit strings of length $n - 2$ with no two consecutive 0s with 10 added at the end. Consequently, there are a_{n-2} such bit strings.

We conclude, as illustrated in Figure 4, that

$$a_n = a_{n-1} + a_{n-2}$$

for $n \geq 3$.

*Reve, more commonly spelled *reeve*, is an archaic word for *governor*.

The initial conditions are $a_1 = 2$, because both bit strings of length one, 0 and 1 do not have consecutive 0s, and $a_2 = 3$, because the valid bit strings of length two are 01, 10, and 11. To obtain a_5 , we use the recurrence relation three times to find that

$$\begin{aligned} a_3 &= a_2 + a_1 = 3 + 2 = 5, \\ a_4 &= a_3 + a_2 = 5 + 3 = 8, \\ a_5 &= a_4 + a_3 = 8 + 5 = 13. \end{aligned}$$

Remark: Note that $\{a_n\}$ satisfies the same recurrence relation as the Fibonacci sequence. Because $a_1 = f_3$ and $a_2 = f_4$ it follows that $a_n = f_{n+2}$.

Example 4 shows how a recurrence relation can be used to model the number of codewords that are allowable using certain validity checks.

EXAMPLE 4 Codeword Enumeration A computer system considers a string of decimal digits a valid codeword if it contains an even number of 0 digits. For instance, 1230407869 is valid, whereas 120987045608 is not valid. Let a_n be the number of valid n -digit codewords. Find a recurrence relation for a_n .

Solution: Note that $a_1 = 9$ because there are 10 one-digit strings, and only one, namely, the string 0, is not valid. A recurrence relation can be derived for this sequence by considering how a valid n -digit string can be obtained from strings of $n - 1$ digits. There are two ways to form a valid string with n digits from a string with one fewer digit.

First, a valid string of n digits can be obtained by appending a valid string of $n - 1$ digits with a digit other than 0. This appending can be done in nine ways. Hence, a valid string with n digits can be formed in this manner in $9a_{n-1}$ ways.

Second, a valid string of n digits can be obtained by appending a 0 to a string of length $n - 1$ that is not valid. (This produces a string with an even number of 0 digits because the invalid string of length $n - 1$ has an odd number of 0 digits.) The number of ways that this can be done equals the number of invalid $(n - 1)$ -digit strings. Because there are 10^{n-1} strings of length $n - 1$, and a_{n-1} are valid, there are $10^{n-1} - a_{n-1}$ valid n -digit strings obtained by appending an invalid string of length $n - 1$ with a 0.

Because all valid strings of length n are produced in one of these two ways, it follows that there are

$$\begin{aligned} a_n &= 9a_{n-1} + (10^{n-1} - a_{n-1}) \\ &= 8a_{n-1} + 10^{n-1} \end{aligned}$$

valid strings of length n .

Example 5 establishes a recurrence relation that appears in many different contexts.

EXAMPLE 5 Find a recurrence relation for C_n , the number of ways to parenthesize the product of $n + 1$ numbers, $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$, to specify the order of multiplication. For example, $C_3 = 5$ because there are five ways to parenthesize $x_0 \cdot x_1 \cdot x_2 \cdot x_3$ to determine the order of multiplication:

$$\begin{aligned} &((x_0 \cdot x_1) \cdot x_2) \cdot x_3 & (x_0 \cdot (x_1 \cdot x_2)) \cdot x_3 & (x_0 \cdot x_1) \cdot (x_2 \cdot x_3) \\ &x_0 \cdot ((x_1 \cdot x_2) \cdot x_3) & x_0 \cdot (x_1 \cdot (x_2 \cdot x_3)). \end{aligned}$$

Solution: To develop a recurrence relation for C_n , we note that however we insert parentheses in the product $x_0 \cdot x_1 \cdot x_2 \cdot \dots \cdot x_n$, one “ $\cdot$ ” operator remains outside all parentheses, namely, the operator for the final multiplication to be performed. [For example, in $(x_0 \cdot (x_1 \cdot x_2)) \cdot x_3$, it is the final “ $\cdot$ ”, while in $(x_0 \cdot x_1) \cdot (x_2 \cdot x_3)$ it is the second “ $\cdot$ ”.] This final operator appears between two of the $n + 1$ numbers, say, x_k and x_{k+1} . There are $C_k C_{n-k-1}$ ways to insert parentheses to determine the order of the $n + 1$ numbers to be multiplied when the final operator appears between x_k and x_{k+1} , because there are C_k ways to insert parentheses in the product $x_0 \cdot x_1 \cdot \dots \cdot x_k$ to determine the order in which these $k + 1$ numbers are to be multiplied and C_{n-k-1} ways to insert parentheses in the product $x_{k+1} \cdot x_{k+2} \cdot \dots \cdot x_n$ to determine the order in which these $n - k$ numbers are to be multiplied. Because this final operator can appear between any two of the $n + 1$ numbers, it follows that

$$\begin{aligned} C_n &= C_0 C_{n-1} + C_1 C_{n-2} + \dots + C_{n-2} C_1 + C_{n-1} C_0 \\ &= \sum_{k=0}^{n-1} C_k C_{n-k-1}. \end{aligned}$$

Note that the initial conditions are $C_0 = 1$ and $C_1 = 1$. 

Links 

The recurrence relation in Example 5 can be solved using the method of generating functions, which will be discussed in Section 8.4. It can be shown that $C_n = C(2n, n)/(n + 1)$ (see Exercise 43 in Section 8.4) and that $C_n \sim \frac{4^n}{n^{3/2} \sqrt{\pi}}$ (see [GrKnPa94]). The sequence $\{C_n\}$ is the sequence of **Catalan numbers**, named after Eugène Charles Catalan. This sequence appears as the solution of many different counting problems besides the one considered here (see the chapter on Catalan numbers in [MiRo91] or [RoTe03] for details).

8.1.3 Algorithms and Recurrence Relations

Recurrence relations play an important role in many aspects of the study of algorithms and their complexity. In Section 8.3, we will show how recurrence relations can be used to analyze the complexity of divide-and-conquer algorithms, such as the merge sort algorithm introduced in Section 5.4. As we will see in Section 8.3, divide-and-conquer algorithms recursively divide a problem into a fixed number of nonoverlapping subproblems until they become simple enough to solve directly. We conclude this section by introducing another algorithmic paradigm known as **dynamic programming**, which can be used to solve many optimization problems efficiently.

Links 

An algorithm follows the dynamic programming paradigm when it recursively breaks down a problem into simpler overlapping subproblems, and computes the solution using the solutions of the subproblems. Generally, recurrence relations are used to find the overall solution from the solutions of the subproblems. Dynamic programming has been used to solve important problems in such diverse areas as economics, computer vision, speech recognition, artificial intelligence, computer graphics, and bioinformatics. In this section we will illustrate the use of dynamic programming by constructing an algorithm for solving a scheduling problem. Before doing so, we will relate the amusing origin of the name *dynamic programming*, which was introduced by the mathematician Richard Bellman in the 1950s. Bellman was working at the RAND Corporation on projects for the U.S. military, and at that time, the U.S. Secretary of Defense was hostile to mathematical research. Bellman decided that to ensure funding, he needed a name not containing the word mathematics for his method for solving scheduling and planning problems. He decided to use the adjective *dynamic* because, as he said “it’s impossible to use the word dynamic in a pejorative sense” and he thought that dynamic programming was “something not even a Congressman could object to.”

AN EXAMPLE OF DYNAMIC PROGRAMMING The problem we use to illustrate dynamic programming is related to the problem studied in Example 7 in Section 3.1. In that problem our goal was to schedule as many talks as possible in a single lecture hall. These talks have preset start and end times; once a talk starts, it continues until it ends; no two talks can proceed at the same time; and a talk can begin at the same time another one ends. We developed a greedy algorithm that always produces an optimal schedule, as we proved in Example 12 in Section 5.1. Now suppose that our goal is not to schedule the most talks possible, but rather to have the largest possible combined attendance of the scheduled talks.

We formalize this problem by supposing that we have n talks, where talk j begins at time t_j , ends at time e_j , and will be attended by w_j students. We want a schedule that maximizes the total number of student attendees. That is, we wish to schedule a subset of talks to maximize the sum of w_j over all scheduled talks. (Note that when a student attends more than one talk, this student is counted according to the number of talks attended.) We denote by $T(j)$ the maximum number of total attendees for an optimal schedule from the first j talks, so $T(n)$ is the maximal number of total attendees for an optimal schedule for all n talks.

We first sort the talks in order of increasing end time. After doing this, we renumber the talks so that $e_1 \leq e_2 \leq \dots \leq e_n$. We say that two talks are **compatible** if they can be part of the same schedule, that is, if the times they are scheduled do not overlap (other than the possibility one ends and the other starts at the same time). We define $p(j)$ to be largest integer i , $i < j$, for which $e_i \leq s_j$, if such an integer exists, and $p(j) = 0$ otherwise. That is, talk $p(j)$ is the talk ending latest among talks compatible with talk j that end before talk j ends, if such a talk exists, and $p(j) = 0$ if there are no such talks.

EXAMPLE 6 Consider seven talks with these start times and end times, as illustrated in Figure 5.

Talk 1: start 8 A.M., end 10 A.M.
 Talk 2: start 9 A.M., end 11 A.M.
 Talk 3: start 10:30 A.M., end 12 noon
 Talk 4: start 9:30 A.M., end 1 P.M.

Talk 5: start 8:30 A.M., end 2 P.M.
 Talk 6: start 11 A.M., end 2 P.M.
 Talk 7: start 1 P.M., end 2 P.M.

Find $p(j)$ for $j = 1, 2, \dots, 7$.

Solution: We have $p(1) = 0$ and $p(2) = 0$, because no talks end before either of the first two talks begin. We have $p(3) = 1$ because talk 3 and talk 1 are compatible, but talk 3 and talk 2 are not compatible; $p(4) = 0$ because talk 4 is not compatible with any of talks 1, 2, and 3; $p(5) = 0$

Links



©Paul Fearn/Alamy Stock Photo

EUGÈNE CHARLES CATALAN (1814–1894) Eugène Catalan was born in Bruges, then part of France. His father became a successful architect in Paris while Eugène was a boy. Catalan attended a Parisian school for design hoping to follow in his father's footsteps. At 15, he won the job of teaching geometry to his design school classmates. After graduating, Catalan attended a school for the fine arts, but because of his mathematical aptitude his instructors recommended that he enter the École Polytechnique. He became a student there, but after his first year, he was expelled because of his politics. However, he was readmitted, and in 1835, he graduated and won a position at the Collège de Châlons sur Marne.

In 1838, Catalan returned to Paris where he founded a preparatory school with two other mathematicians, Sturm and Liouville. After teaching there for a short time, he was appointed to a position at the École Polytechnique. He received his doctorate from the École Polytechnique in 1841, but his political activity in favor of the French Republic hurt his career prospects. In 1846 Catalan held a position at the Collège de Charlemagne; he was appointed to the Lycée Saint Louis in 1849. However, when Catalan would not take a required oath of allegiance to the new Emperor Louis-Napoleon Bonaparte, he lost his job. For 13 years he held no permanent position. Finally, in 1865 he was appointed to a chair of mathematics at the University of Liège, Belgium, a position he held until his 1884 retirement.

Catalan made many contributions to number theory and to the related subject of continued fractions. He defined what are now known as the Catalan numbers when he solved the problem of dissecting a polygon into triangles using non-intersecting diagonals. Catalan is also well known for formulating what was known as the *Catalan conjecture*. This asserted that 8 and 9 are the only consecutive powers of integers, a conjecture not solved until 2003. Catalan wrote many textbooks, including several that became quite popular and appeared in as many as 12 editions. Perhaps this textbook will have a 12th edition someday!

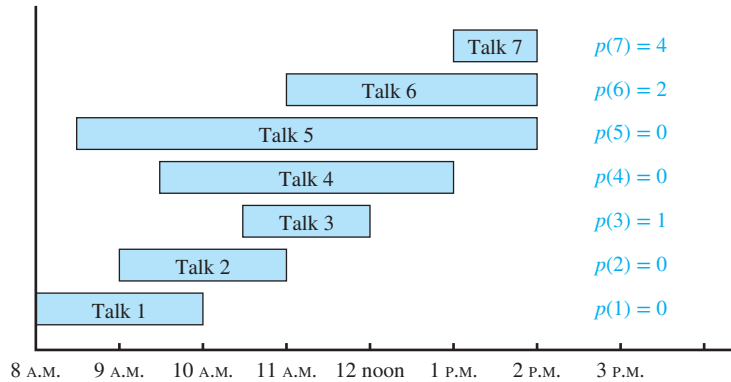


FIGURE 5 A schedule of lectures with the values of $p(n)$ shown.

because talk 5 is not compatible with any of talks 1, 2, 3, and 4; and $p(6) = 2$ because talk 6 and talk 2 are compatible, but talk 6 is not compatible with any of talks 3, 4, and 5. Finally, $p(7) = 4$, because talk 7 and talk 4 are compatible, but talk 7 is not compatible with either of talks 5 or 6. ◀

To develop a dynamic programming algorithm for this problem, we first develop a key recurrence relation. To do this, first note that if $j \leq n$, there are two possibilities for an optimal schedule of the first j talks (recall that we are assuming that the n talks are ordered by increasing end time): (i) talk j belongs to the optimal schedule or (ii) it does not.

Case (i): We know that talks $p(j) + 1, \dots, j - 1$ do not belong to this schedule, for none of these other talks are compatible with talk j . Furthermore, the other talks in this optimal schedule must comprise an optimal schedule for talks 1, 2, $\dots, p(j)$. For if there were a better schedule for talks

Links



©Alfred Eisenstaedt/The LIFE Picture Collection/Getty Images

RICHARD BELLMAN (1920–1984) Richard Bellman, born in Brooklyn, where his father was a grocer, spent many hours in the museums and libraries of New York as a child. After graduating high school, he studied mathematics at Brooklyn College and graduated in 1941. He began postgraduate work at Johns Hopkins University, but because of the war, left to teach electronics at the University of Wisconsin. He was able to continue his mathematics studies at Wisconsin, and in 1943 he received his masters degree there. Later, Bellman entered Princeton University, teaching in a special U.S. Army program. In late 1944, he was drafted into the army. He was assigned to the Manhattan Project at Los Alamos where he worked in theoretical physics. After the war, he returned to Princeton and received his Ph.D. in 1946.

After briefly teaching at Princeton, he moved to Stanford University, where he attained tenure. At Stanford he pursued his fascination with number theory. However, Bellman decided to focus on mathematical questions arising from real-world problems. In 1952, he joined the RAND Corporation, working on multistage decision processes, operations research problems, and applications to the social sciences and medicine. He worked on many military projects while at RAND. In 1965 he left RAND to become professor of mathematics, electrical and biomedical engineering and medicine at the University of Southern California.

In the 1950s Bellman pioneered the use of dynamic programming, a technique invented earlier, in a wide range of settings. He is also known for his work on stochastic control processes, in which he introduced what is now called the Bellman equation. He coined the term *curse of dimensionality* to describe problems caused by the exponential increase in volume associated with adding extra dimensions to a space. He wrote an amazing number of books and research papers with many coauthors, including many on industrial production and economic systems. His work led to the application of computing techniques in a wide variety of areas ranging from the design of guidance systems for space vehicles, to network optimization, and even to pest control.

Tragically, in 1973 Bellman was diagnosed with a brain tumor. Although it was removed successfully, complications left him severely disabled. Fortunately, he managed to continue his research and writing during his remaining ten years of life. Bellman received many prizes and awards, including the first Norbert Wiener Prize in Applied Mathematics and the IEEE Gold Medal of Honor. He was elected to the National Academy of Sciences. He was held in high regard for his achievements, courage, and admirable qualities. Bellman was the father of two children.

1, 2, ..., $p(j)$, by adding talk j , we will have a schedule better than the overall optimal schedule. Consequently, in case (i), we have $T(j) = w_j + T(p(j))$.

Case (ii): When talk j does not belong to an optimal schedule, it follows that an optimal schedule from talks 1, 2, ..., j is the same as an optimal schedule from talks 1, 2, ..., $j - 1$. Consequently, in case (ii), we have $T(j) = T(j - 1)$. Combining cases (i) and (ii) leads us to the recurrence relation

$$T(j) = \max(w_j + T(p(j)), T(j - 1)).$$

Now that we have developed this recurrence relation, we can construct an efficient algorithm, Algorithm 1, for computing the maximum total number of attendees. We ensure that the algorithm is efficient by storing the value of each $T(j)$ after we compute it. This allows us to compute $T(j)$ only once. If we did not do this, the algorithm would have exponential worst-case complexity. The process of storing the values as each is computed is known as **memoization** and is an important technique for making recursive algorithms efficient.

ALGORITHM 1 Dynamic Programming Algorithm for Scheduling Talks.

```

procedure Maximum Attendees ( $s_1, s_2, \dots, s_n$ : start times of talks;
     $e_1, e_2, \dots, e_n$ : end times of talks;  $w_1, w_2, \dots, w_n$ : number of attendees to talks)
    sort talks by end time and relabel so that  $e_1 \leq e_2 \leq \dots \leq e_n$ 
    for  $j := 1$  to  $n$ 
        if no job  $i$  with  $i < j$  is compatible with job  $j$ 
             $p(j) = 0$ 
        else  $p(j) := \max\{i \mid i < j \text{ and job } i \text{ is compatible with job } j\}$ 
         $T(0) := 0$ 
    for  $j := 1$  to  $n$ 
         $T(j) := \max(w_j + T(p(j)), T(j - 1))$ 
    return  $T(n)$  { $T(n)$  is the maximum number of attendees}

```

In Algorithm 1 we determine the maximum number of attendees that can be achieved by a schedule of talks, but we do not find a schedule that achieves this maximum. To find talks we need to schedule, we use the fact that talk j belongs to an optimal solution for the first j talks if and only if $w_j + T(p(j)) \geq T(j - 1)$. We leave it as Exercise 53 to construct an algorithm based on this observation that determines which talks should be scheduled to achieve the maximum total number of attendees.

Algorithm 1 is a good example of dynamic programming as the maximum total attendance is found using the optimal solutions of the overlapping subproblems, each of which determines the maximum total attendance of the first j talks for some j with $1 \leq j \leq n - 1$. See Exercises 56 and 57 and Supplementary Exercises 14 and 17 for other examples of dynamic programming.

Exercises

1. Use mathematical induction to verify the formula derived in Example 2 for the number of moves required to complete the Tower of Hanoi puzzle.
2. a) Find a recurrence relation for the number of permutations of a set with n elements.
b) Use this recurrence relation to find the number of permutations of a set with n elements using iteration.
3. A vending machine dispensing books of stamps accepts only one-dollar coins, \$1 bills, and \$5 bills.
 - a) Find a recurrence relation for the number of ways to deposit n dollars in the vending machine, where the order in which the coins and bills are deposited matters.

- b) What are the initial conditions?
 c) How many ways are there to deposit \$10 for a book of stamps?
4. A country uses as currency coins with values of 1 peso, 2 pesos, 5 pesos, and 10 pesos and bills with values of 5 pesos, 10 pesos, 20 pesos, 50 pesos, and 100 pesos. Find a recurrence relation for the number of ways to pay a bill of n pesos if the order in which the coins and bills are paid matters.
5. How many ways are there to pay a bill of 17 pesos using the currency described in Exercise 4, where the order in which coins and bills are paid matters?
- *6. a) Find a recurrence relation for the number of strictly increasing sequences of positive integers that have 1 as their first term and n as their last term, where n is a positive integer. That is, sequences $a_1, a_2, \dots, a_k$, where $a_1 = 1$, $a_k = n$, and $a_j < a_{j+1}$ for $j = 1, 2, \dots, k-1$.
 b) What are the initial conditions?
 c) How many sequences of the type described in (a) are there when n is an integer with $n \geq 2$?
7. a) Find a recurrence relation for the number of bit strings of length n that contain a pair of consecutive 0s.
 b) What are the initial conditions?
 c) How many bit strings of length seven contain two consecutive 0s?
8. a) Find a recurrence relation for the number of bit strings of length n that contain three consecutive 0s.
 b) What are the initial conditions?
 c) How many bit strings of length seven contain three consecutive 0s?
9. a) Find a recurrence relation for the number of bit strings of length n that do not contain three consecutive 0s.
 b) What are the initial conditions?
 c) How many bit strings of length seven do not contain three consecutive 0s?
- *10. a) Find a recurrence relation for the number of bit strings of length n that contain the string 01.
 b) What are the initial conditions?
 c) How many bit strings of length seven contain the string 01?
11. a) Find a recurrence relation for the number of ways to climb n stairs if the person climbing the stairs can take one stair or two stairs at a time.
 b) What are the initial conditions?
 c) In how many ways can this person climb a flight of eight stairs?
12. a) Find a recurrence relation for the number of ways to climb n stairs if the person climbing the stairs can take one, two, or three stairs at a time.
 b) What are the initial conditions?
 c) In how many ways can this person climb a flight of eight stairs?
13. a) Find a recurrence relation for the number of ternary strings of length n that do not contain two consecutive 0s.
 b) What are the initial conditions?
 c) How many ternary strings of length six do not contain two consecutive 0s?
14. a) Find a recurrence relation for the number of ternary strings of length n that contain two consecutive 0s.
 b) What are the initial conditions?
 c) How many ternary strings of length six contain two consecutive 0s?
- *15. a) Find a recurrence relation for the number of ternary strings of length n that do not contain two consecutive 0s or two consecutive 1s.
 b) What are the initial conditions?
 c) How many ternary strings of length six do not contain two consecutive 0s or two consecutive 1s?
- *16. a) Find a recurrence relation for the number of ternary strings of length n that contain either two consecutive 0s or two consecutive 1s.
 b) What are the initial conditions?
 c) How many ternary strings of length six contain two consecutive 0s or two consecutive 1s?
- *17. a) Find a recurrence relation for the number of ternary strings of length n that do not contain consecutive symbols that are the same.
 b) What are the initial conditions?
 c) How many ternary strings of length six do not contain consecutive symbols that are the same?
- **18. a) Find a recurrence relation for the number of ternary strings of length n that contain two consecutive symbols that are the same.
 b) What are the initial conditions?
 c) How many ternary strings of length six contain consecutive symbols that are the same?
19. Messages are transmitted over a communications channel using two signals. The transmittal of one signal requires 1 microsecond, and the transmittal of the other signal requires 2 microseconds.
 a) Find a recurrence relation for the number of different messages consisting of sequences of these two signals, where each signal in the message is immediately followed by the next signal, that can be sent in n microseconds.
 b) What are the initial conditions?
 c) How many different messages can be sent in 10 microseconds using these two signals?
20. A bus driver pays all tolls, using only nickels and dimes, by throwing one coin at a time into the mechanical toll collector.
 a) Find a recurrence relation for the number of different ways the bus driver can pay a toll of n cents (where the order in which the coins are used matters).
 b) In how many different ways can the driver pay a toll of 45 cents?
21. a) Find the recurrence relation satisfied by R_n , where R_n is the number of regions that a plane is divided into by n lines, if no two of the lines are parallel and no three of the lines go through the same point.
 b) Find R_n using iteration.

A string that contains only 0s, 1s, and 2s is called a **ternary string**.

- *22. a) Find the recurrence relation satisfied by R_n , where R_n is the number of regions into which the surface of a sphere is divided by n great circles (which are the intersections of the sphere and planes passing through the center of the sphere), if no three of the great circles go through the same point.
b) Find R_n using iteration.
- *23. a) Find the recurrence relation satisfied by S_n , where S_n is the number of regions into which three-dimensional space is divided by n planes if every three of the planes meet in one point, but no four of the planes go through the same point.
b) Find S_n using iteration.
24. Find a recurrence relation for the number of bit sequences of length n with an even number of 0s.
25. How many bit sequences of length seven contain an even number of 0s?
26. a) Find a recurrence relation for the number of ways to completely cover a $2 \times n$ checkerboard with 1×2 dominoes. [Hint: Consider separately the coverings where the position in the top right corner of the checkerboard is covered by a domino positioned horizontally and where it is covered by a domino positioned vertically.]
b) What are the initial conditions for the recurrence relation in part (a)?
c) How many ways are there to completely cover a 2×17 checkerboard with 1×2 dominoes?
27. a) Find a recurrence relation for the number of ways to lay out a walkway with slate tiles if the tiles are red, green, or gray, so that no two red tiles are adjacent and tiles of the same color are considered indistinguishable.
b) What are the initial conditions for the recurrence relation in part (a)?
c) How many ways are there to lay out a path of seven tiles as described in part (a)?
28. Show that the Fibonacci numbers satisfy the recurrence relation $f_n = 5f_{n-4} + 3f_{n-5}$ for $n = 5, 6, 7, \dots$, together with the initial conditions $f_0 = 0, f_1 = 1, f_2 = 1, f_3 = 2$, and $f_4 = 3$. Use this recurrence relation to show that f_{5n} is divisible by 5, for $n = 1, 2, 3, \dots$.
- *29. Let $S(m, n)$ denote the number of onto functions from a set with m elements to a set with n elements. Show that $S(m, n)$ satisfies the recurrence relation

$$S(m, n) = n^m - \sum_{k=1}^{n-1} C(n, k)S(m, k)$$

whenever $m \geq n$ and $n > 1$, with the initial condition $S(m, 1) = 1$.

30. a) Write out all the ways the product $x_0 \cdot x_1 \cdot x_2 \cdot x_3 \cdot x_4$ can be parenthesized to determine the order of multiplication.
b) Use the recurrence relation developed in Example 5 to calculate C_4 , the number of ways to parenthesize the product of five numbers so as to determine the order of multiplication. Verify that you listed the correct number of ways in part (a).
- c) Check your result in part (b) by finding C_4 , using the closed formula for C_n mentioned in the solution of Example 5.
31. a) Use the recurrence relation developed in Example 5 to determine C_5 , the number of ways to parenthesize the product of six numbers so as to determine the order of multiplication.
b) Check your result with the closed formula for C_5 mentioned in the solution of Example 5.
- *32. In the Tower of Hanoi puzzle, suppose our goal is to transfer all n disks from peg 1 to peg 3, but we cannot move a disk directly between pegs 1 and 3. Each move of a disk must be a move involving peg 2. As usual, we cannot place a disk on top of a smaller disk.
- a) Find a recurrence relation for the number of moves required to solve the puzzle for n disks with this added restriction.
b) Solve this recurrence relation to find a formula for the number of moves required to solve the puzzle for n disks.
c) How many different arrangements are there of the n disks on three pegs so that no disk is on top of a smaller disk?
d) Show that every allowable arrangement of the n disks occurs in the solution of this variation of the puzzle.

Exercises 33–37 deal with a variation of the **Josephus problem** described by Graham, Knuth, and Patashnik in [GrKnPa94]. This problem is based on an account by the historian Flavius Josephus, who was part of a band of 41 Jewish rebels trapped in a cave by the Romans during the Jewish-Roman war of the first century. The rebels preferred suicide to capture; they decided to form a circle and to repeatedly count off around the circle, killing every third rebel left alive. However, Josephus and another rebel did not want to be killed this way; they determined the positions where they should stand to be the last two rebels remaining alive. The variation we consider begins with n people, numbered 1 to n , standing around a circle. In each stage, every second person still left alive is eliminated until only one survives. We denote the number of the survivor by $J(n)$.

33. Determine the value of $J(n)$ for each integer n with $1 \leq n \leq 16$.
34. Use the values you found in Exercise 33 to conjecture a formula for $J(n)$. [Hint: Write $n = 2^m + k$, where m is a nonnegative integer and k is a nonnegative integer less than 2^m .]
35. Show that $J(n)$ satisfies the recurrence relation $J(2n) = 2J(n) - 1$ and $J(2n + 1) = 2J(n) + 1$, for $n \geq 1$, and $J(1) = 1$.
36. Use mathematical induction to prove the formula you conjectured in Exercise 34, making use of the recurrence relation from Exercise 35.

37. Determine $J(100)$, $J(1000)$, and $J(10,000)$ from your formula for $J(n)$.

Exercises 38–45 involve the Reve's puzzle, the variation of the Tower of Hanoi puzzle with four pegs and n disks. Before presenting these exercises, we describe the Frame–Stewart algorithm for moving the disks from peg 1 to peg 4 so that no disk is ever on top of a smaller one. This algorithm, given the number of disks n as input, depends on a choice of an integer k with $1 \leq k \leq n$. When there is only one disk, move it from peg 1 to peg 4 and stop. For $n > 1$, the algorithm proceeds recursively, using these three steps. Recursively move the stack of the $n - k$ smallest disks from peg 1 to peg 2, using all four pegs. Next move the stack of the k largest disks from peg 1 to peg 4, using the three-peg algorithm from the Tower of Hanoi puzzle without using the peg holding the $n - k$ smallest disks. Finally, recursively move the smallest $n - k$ disks to peg 4, using all four pegs. Frame and Stewart showed that to produce the fewest moves using their algorithm, k should be chosen to be the smallest integer such that n does not exceed $t_k = k(k + 1)/2$, the k th triangular number, that is, $t_{k-1} < n \leq t_k$. The long-standing conjecture, known as **Frame's conjecture**, that this algorithm uses the fewest number of moves required to solve the puzzle, was proved by Thierry Bousch in 2014.

38. Show that the Reve's puzzle with three disks can be solved using five, and no fewer, moves.
39. Show that the Reve's puzzle with four disks can be solved using nine, and no fewer, moves.
40. Describe the moves made by the Frame–Stewart algorithm, with k chosen so that the fewest moves are required, for
- a) 5 disks. b) 6 disks. c) 7 disks. d) 8 disks.
- *41. Show that if $R(n)$ is the number of moves used by the Frame–Stewart algorithm to solve the Reve's puzzle with n disks, where k is chosen to be the smallest integer with $n \leq k(k + 1)/2$, then $R(n)$ satisfies the recurrence relation $R(n) = 2R(n - k) + 2^k - 1$, with $R(0) = 0$ and $R(1) = 1$.
- *42. Show that if k is as chosen in Exercise 41, then $R(n) - R(n - 1) = 2^{k-1}$.
- *43. Show that if k is as chosen in Exercise 41, then $R(n) = \sum_{i=1}^k i2^{i-1} - (t_k - n)2^{k-1}$.
- *44. Use Exercise 43 to give an upper bound on the number of moves required to solve the Reve's puzzle for all integers n with $1 \leq n \leq 25$.
- *45. Show that $R(n)$ is $O(\sqrt{n}2^{\sqrt{2n}})$.

Let $\{a_n\}$ be a sequence of real numbers. The **backward differences** of this sequence are defined recursively as shown next. The **first difference** ∇a_n is

$$\nabla a_n = a_n - a_{n-1}.$$

The **$(k + 1)$ st difference** $\nabla^{k+1}a_n$ is obtained from $\nabla^k a_n$ by

$$\nabla^{k+1}a_n = \nabla^k a_n - \nabla^k a_{n-1}.$$

46. Find ∇a_n for the sequence $\{a_n\}$, where
- a) $a_n = 4$. b) $a_n = 2n$.
c) $a_n = n^2$. d) $a_n = 2^n$.
47. Find $\nabla^2 a_n$ for the sequences in Exercise 46.
48. Show that $a_{n-1} = a_n - \nabla a_n$.
49. Show that $a_{n-2} = a_n - 2\nabla a_n + \nabla^2 a_n$.
- *50. Prove that a_{n-k} can be expressed in terms of a_n , ∇a_n , $\nabla^2 a_n$, ..., $\nabla^k a_n$.
51. Express the recurrence relation $a_n = a_{n-1} + a_{n-2}$ in terms of a_n , ∇a_n , and $\nabla^2 a_n$.
52. Show that any recurrence relation for the sequence $\{a_n\}$ can be written in terms of a_n , ∇a_n , $\nabla^2 a_n$, The resulting equation involving the sequences and its differences is called a **difference equation**.
- *53. Construct the algorithm described in the text after Algorithm 1 for determining which talks should be scheduled to maximize the total number of attendees and not just the maximum total number of attendees determined by Algorithm 1.
54. Use Algorithm 1 to determine the maximum number of total attendees in the talks in Example 6 if w_i , the number of attendees of talk i , $i = 1, 2, \dots, 7$, is
- a) 20, 10, 50, 30, 15, 25, 40.
b) 100, 5, 10, 20, 25, 40, 30.
c) 2, 3, 8, 5, 4, 7, 10.
d) 10, 8, 7, 25, 20, 30, 5.
55. For each part of Exercise 54, use your algorithm from Exercise 53 to find the optimal schedule for talks so that the total number of attendees is maximized.
56. In this exercise we will develop a dynamic programming algorithm for finding the maximum sum of consecutive terms of a sequence of real numbers. That is, given a sequence of real numbers $a_1, a_2, \dots, a_n$, the algorithm computes the maximum sum $\sum_{i=j}^k a_i$ where $1 \leq j \leq k \leq n$.
- a) Show that if all terms of the sequence are nonnegative, this problem is solved by taking the sum of all terms. Then, give an example where the maximum sum of consecutive terms is not the sum of all terms.
- b) Let $M(k)$ be the maximum of the sums of consecutive terms of the sequence ending at a_k . That is, $M(k) = \max_{1 \leq j \leq k} \sum_{i=j}^k a_i$. Explain why the recurrence relation $M(k) = \max(M(k - 1) + a_k, a_k)$ holds for $k = 2, \dots, n$.
- c) Use part (b) to develop a dynamic programming algorithm for solving this problem.
- d) Show each step your algorithm from part (c) uses to find the maximum sum of consecutive terms of the sequence 2, -3, 4, 1, -2, 3.
- e) Show that the worst-case complexity in terms of the number of additions and comparisons of your algorithm from part (c) is linear.

*57. Dynamic programming can be used to develop an algorithm for solving the matrix-chain multiplication problem introduced in Section 3.3. This is the problem of determining how the product $A_1 A_2 \cdots A_n$ can be computed using the fewest integer multiplications, where $A_1, A_2, \dots, A_n$ are $m_1 \times m_2, m_2 \times m_3, \dots, m_n \times m_{n+1}$ matrices, respectively, and each matrix has integer entries. Recall that by the associative law, the product does not depend on the order in which the matrices are multiplied.

- a) Show that the brute-force method of determining the minimum number of integer multiplications needed to solve a matrix-chain multiplication problem has exponential worst-case complexity. [Hint: Do this by first showing that the order of multiplication of matrices is specified by parenthesizing the product. Then, use Example 5 and the result of part (c) of Exercise 43 in Section 8.4.]
- b) Denote by A_{ij} the product $A_i A_{i+1} \cdots A_j$, and $M(i, j)$ the minimum number of integer multiplications required to find A_{ij} . Show that if the

least number of integer multiplications are used to compute A_{ij} , where $i < j$, by splitting the product into the product of A_i through A_k and the product of A_{k+1} through A_j , then the first k terms must be parenthesized so that A_{ik} is computed in the optimal way using $M(i, k)$ integer multiplications, and $A_{k+1,j}$ must be parenthesized so that $A_{k+1,j}$ is computed in the optimal way using $M(k+1, j)$ integer multiplications.

- c) Explain why part (b) leads to the recurrence relation $M(i, j) = \min_{i \leq k < j} (M(i, k) + M(k+1, j) + m_i m_{k+1} m_{j+1})$ if $1 \leq i \leq j < n$.
- d) Use the recurrence relation in part (c) to construct an efficient algorithm for determining the order the n matrices should be multiplied to use the minimum number of integer multiplications. Store the partial results $M(i, j)$ as you find them so that your algorithm will not have exponential complexity.
- e) Show that your algorithm from part (d) has $O(n^3)$ worst-case complexity in terms of multiplications of integers.

8.2 Solving Linear Recurrence Relations

8.2.1 Introduction



A wide variety of recurrence relations occur in models. Some of these recurrence relations can be solved using iteration or some other ad hoc technique. However, one important class of recurrence relations can be explicitly solved in a systematic way. These are recurrence relations that express the terms of a sequence as linear combinations of previous terms.

Definition 1

A linear homogeneous recurrence relation of degree k with constant coefficients is a recurrence relation of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

where $c_1, c_2, \dots, c_k$ are real numbers, and $c_k \neq 0$.

The recurrence relation in the definition is **linear** because the right-hand side is a sum of previous terms of the sequence each multiplied by a function of n . The recurrence relation is **homogeneous** because no terms occur that are not multiples of the a_j s. The coefficients of the terms of the sequence are all **constants**, rather than functions that depend on n . The **degree** is k because a_n is expressed in terms of the previous k terms of the sequence.

A consequence of the second principle of mathematical induction is that a sequence satisfying the recurrence relation in the definition is uniquely determined by this recurrence relation and the k initial conditions

$$a_0 = C_0, a_1 = C_1, \dots, a_{k-1} = C_{k-1}.$$

EXAMPLE 1

The recurrence relation $P_n = (1.11)P_{n-1}$ is a linear homogeneous recurrence relation of degree one. The recurrence relation $f_n = f_{n-1} + f_{n-2}$ is a linear homogeneous recurrence relation of

degree two. The recurrence relation $a_n = a_{n-5}$ is a linear homogeneous recurrence relation of degree five. 

To help clarify the definition of linear homogeneous recurrence relations with constant coefficients, we will now provide examples of recurrence relations each lacking one of the defining properties.

EXAMPLE 2 The recurrence relation $a_n = a_{n-1} + a_{n-2}^2$ is not linear. The recurrence relation $H_n = 2H_{n-1} + 1$ is not homogeneous. The recurrence relation $B_n = nB_{n-1}$ does not have constant coefficients. 

Linear homogeneous recurrence relations are studied for two reasons. First, they often occur in modeling of problems. Second, they can be systematically solved.

8.2.2 Solving Linear Homogeneous Recurrence Relations with Constant Coefficients

Recurrence relations may be difficult to solve, but fortunately this is not the case for linear homogeneous recurrence relations with constant coefficients. We can use two key ideas to find all their solutions. First, these recurrence relations have solutions of the form $a_n = r^n$, where r is a constant. To see this, observe that $a_n = r^n$ is a solution of the recurrence relation $a_n = c_1a_{n-1} + c_2a_{n-2} + \cdots + c_ka_{n-k}$ if and only if

$$r^n = c_1r^{n-1} + c_2r^{n-2} + \cdots + c_kr^{n-k}.$$

When both sides of this equation are divided by r^{n-k} (when $r \neq 0$) and the right-hand side is subtracted from the left, we obtain the equation

$$r^k - c_1r^{k-1} - c_2r^{k-2} - \cdots - c_{k-1}r - c_k = 0.$$

Consequently, the sequence $\{a_n\}$ with $a_n = r^n$ where $r \neq 0$ is a solution if and only if r is a solution of this last equation. We call this the **characteristic equation** of the recurrence relation. The solutions of this equation are called the **characteristic roots** of the recurrence relation. As we will see, these characteristic roots can be used to give an explicit formula for all the solutions of the recurrence relation.

The other key observation is that a linear combination of two solutions of a linear homogeneous recurrence relation is also a solution. To see this, suppose that s_n and t_n are both solutions of $a_n = c_1a_{n-1} + c_2a_{n-2} + \cdots + c_ka_{n-k}$. Then

$$s_n = c_1s_{n-1} + c_2s_{n-2} + \cdots + c_ks_{n-k}$$

and

$$t_n = c_1t_{n-1} + c_2t_{n-2} + \cdots + c_kt_{n-k}.$$

Now suppose that b_1 and b_2 are real numbers. Then

$$\begin{aligned} b_1s_n + b_2t_n &= b_1(c_1s_{n-1} + c_2s_{n-2} + \cdots + c_ks_{n-k}) + b_2(c_1t_{n-1} + c_2t_{n-2} + \cdots + c_kt_{n-k}) \\ &= c_1(b_1s_{n-1} + b_2t_{n-1}) + c_2(b_1s_{n-2} + b_2t_{n-2}) + \cdots + c_k(b_1s_{n-k} + b_2t_{n-k}). \end{aligned}$$

This means that $b_1s_n + b_2t_n$ is also a solution of the same linear homogeneous recurrence relation.

Using these key observations, we will show how to solve linear homogeneous recurrence relations with constant coefficients.

THE DEGREE TWO CASE We now turn our attention to linear homogeneous recurrence relations of degree two. First, consider the case when there are two distinct characteristic roots.

THEOREM 1

Let c_1 and c_2 be real numbers. Suppose that $r^2 - c_1r - c_2 = 0$ has two distinct roots r_1 and r_2 . Then the sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = c_1a_{n-1} + c_2a_{n-2}$ if and only if $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ for $n = 0, 1, 2, \dots$, where α_1 and α_2 are constants.

Proof: We must do two things to prove the theorem. First, it must be shown that if r_1 and r_2 are the roots of the characteristic equation, and α_1 and α_2 are constants, then the sequence $\{a_n\}$ with $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ is a solution of the recurrence relation. Second, it must be shown that if the sequence $\{a_n\}$ is a solution, then $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ for some constants α_1 and α_2 .

We now show that if $a_n = \alpha_1r_1^n + \alpha_2r_2^n$, then the sequence $\{a_n\}$ is a solution of the recurrence relation. Because r_1 and r_2 are roots of $r^2 - c_1r - c_2 = 0$, it follows that $r_1^2 = c_1r_1 + c_2$ and $r_2^2 = c_1r_2 + c_2$.

From these equations, we see that

$$\begin{aligned} c_1a_{n-1} + c_2a_{n-2} &= c_1(\alpha_1r_1^{n-1} + \alpha_2r_2^{n-1}) + c_2(\alpha_1r_1^{n-2} + \alpha_2r_2^{n-2}) \\ &= \alpha_1r_1^{n-2}(c_1r_1 + c_2) + \alpha_2r_2^{n-2}(c_1r_2 + c_2) \\ &= \alpha_1r_1^{n-2}r_1^2 + \alpha_2r_2^{n-2}r_2^2 \\ &= \alpha_1r_1^n + \alpha_2r_2^n \\ &= a_n. \end{aligned}$$

This shows that the sequence $\{a_n\}$ with $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ is a solution of the recurrence relation.

To show that every solution $\{a_n\}$ of the recurrence relation $a_n = c_1a_{n-1} + c_2a_{n-2}$ has $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ for $n = 0, 1, 2, \dots$, for some constants α_1 and α_2 , suppose that $\{a_n\}$ is a solution of the recurrence relation, and the initial conditions $a_0 = C_0$ and $a_1 = C_1$ hold. It will be shown that there are constants α_1 and α_2 such that the sequence $\{a_n\}$ with $a_n = \alpha_1r_1^n + \alpha_2r_2^n$ satisfies these same initial conditions. This requires that

$$\begin{aligned} a_0 &= C_0 = \alpha_1 + \alpha_2, \\ a_1 &= C_1 = \alpha_1r_1 + \alpha_2r_2. \end{aligned}$$

We can solve these two equations for α_1 and α_2 . From the first equation it follows that $\alpha_2 = C_0 - \alpha_1$. Inserting this expression into the second equation gives

$$C_1 = \alpha_1r_1 + (C_0 - \alpha_1)r_2.$$

Hence,

$$C_1 = \alpha_1(r_1 - r_2) + C_0r_2.$$

This shows that

$$\alpha_1 = \frac{C_1 - C_0r_2}{r_1 - r_2}$$

and

$$\alpha_2 = C_0 - \alpha_1 = C_0 - \frac{C_1 - C_0r_2}{r_1 - r_2} = \frac{C_0r_1 - C_1}{r_1 - r_2},$$

where these expressions for α_1 and α_2 depend on the fact that $r_1 \neq r_2$. (When $r_1 = r_2$, this theorem is not true.) Hence, with these values for α_1 and α_2 , the sequence $\{a_n\}$ with $\alpha_1 r_1^n + \alpha_2 r_2^n$ satisfies the two initial conditions.

We know that $\{a_n\}$ and $\{\alpha_1 r_1^n + \alpha_2 r_2^n\}$ are both solutions of the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ and both satisfy the initial conditions when $n = 0$ and $n = 1$. Because there is a unique solution of a linear homogeneous recurrence relation of degree two with two initial conditions, it follows that the two solutions are the same, that is, $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$ for all nonnegative integers n . We have completed the proof by showing that a solution of the linear homogeneous recurrence relation with constant coefficients of degree two must be of the form $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$, where α_1 and α_2 are constants. 

The characteristic roots of a linear homogeneous recurrence relation with constant coefficients may be complex numbers. Theorem 1 (and also subsequent theorems in this section) still applies in this case. Recurrence relations with complex characteristic roots will not be discussed in the text. Readers familiar with complex numbers may wish to solve Exercises 38 and 39.

Examples 3 and 4 show how to use Theorem 1 to solve recurrence relations.

EXAMPLE 3 What is the solution of the recurrence relation

Extra Examples 

$$a_n = a_{n-1} + 2a_{n-2}$$

with $a_0 = 2$ and $a_1 = 7$?

Solution: Theorem 1 can be used to solve this problem. The characteristic equation of the recurrence relation is $r^2 - r - 2 = 0$. Its roots are $r = 2$ and $r = -1$. Hence, the sequence $\{a_n\}$ is a solution to the recurrence relation if and only if

$$a_n = \alpha_1 2^n + \alpha_2 (-1)^n,$$

for some constants α_1 and α_2 . From the initial conditions, it follows that

$$\begin{aligned} a_0 = 2 &= \alpha_1 + \alpha_2, \\ a_1 = 7 &= \alpha_1 \cdot 2 + \alpha_2 \cdot (-1). \end{aligned}$$

Solving these two equations shows that $\alpha_1 = 3$ and $\alpha_2 = -1$. Hence, the solution to the recurrence relation and initial conditions is the sequence $\{a_n\}$ with

$$a_n = 3 \cdot 2^n - (-1)^n. \quad \text{◀}$$

EXAMPLE 4 Find an explicit formula for the Fibonacci numbers.

Solution: Recall that the sequence of Fibonacci numbers satisfies the recurrence relation $f_n = f_{n-1} + f_{n-2}$ and also satisfies the initial conditions $f_0 = 0$ and $f_1 = 1$. The roots of the characteristic equation $r^2 - r - 1 = 0$ are $r_1 = (1 + \sqrt{5})/2$ and $r_2 = (1 - \sqrt{5})/2$. Therefore, from Theorem 1 it follows that the Fibonacci numbers are given by

$$f_n = \alpha_1 \left(\frac{1 + \sqrt{5}}{2} \right)^n + \alpha_2 \left(\frac{1 - \sqrt{5}}{2} \right)^n,$$

for some constants α_1 and α_2 . The initial conditions $f_0 = 0$ and $f_1 = 1$ can be used to find these constants. We have

$$\begin{aligned} f_0 &= \alpha_1 + \alpha_2 = 0, \\ f_1 &= \alpha_1 \left(\frac{1 + \sqrt{5}}{2} \right) + \alpha_2 \left(\frac{1 - \sqrt{5}}{2} \right) = 1. \end{aligned}$$

The solution to these simultaneous equations for α_1 and α_2 is

$$\alpha_1 = 1/\sqrt{5}, \quad \alpha_2 = -1/\sqrt{5}.$$

Consequently, the Fibonacci numbers are given by

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n.$$

Theorem 1 does not apply when there is one characteristic root of multiplicity two. If this happens, then $a_n = nr_0^n$ is another solution of the recurrence relation when r_0 is a root of multiplicity two of the characteristic equation. Theorem 2 shows how to handle this case.

THEOREM 2

Let c_1 and c_2 be real numbers with $c_2 \neq 0$. Suppose that $r^2 - c_1r - c_2 = 0$ has only one root r_0 . A sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = c_1a_{n-1} + c_2a_{n-2}$ if and only if $a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$, for $n = 0, 1, 2, \dots$, where α_1 and α_2 are constants.

The proof of Theorem 2 is left as Exercise 10. Example 5 illustrates the use of this theorem.

EXAMPLE 5

What is the solution of the recurrence relation

$$a_n = 6a_{n-1} - 9a_{n-2}$$

with initial conditions $a_0 = 1$ and $a_1 = 6$?

Solution: The only root of $r^2 - 6r + 9 = 0$ is $r = 3$. Hence, the solution to this recurrence relation is

$$a_n = \alpha_1 3^n + \alpha_2 n 3^n$$

for some constants α_1 and α_2 . Using the initial conditions, it follows that

$$\begin{aligned} a_0 &= 1 = \alpha_1, \\ a_1 &= 6 = \alpha_1 \cdot 3 + \alpha_2 \cdot 3. \end{aligned}$$

Solving these two equations shows that $\alpha_1 = 1$ and $\alpha_2 = 1$. Consequently, the solution to this recurrence relation and the initial conditions is

$$a_n = 3^n + n3^n.$$

THE GENERAL CASE We will now state the general result about the solution of linear homogeneous recurrence relations with constant coefficients, where the degree may be greater than two, under the assumption that the characteristic equation has distinct roots. The proof of this result will be left as Exercise 16.

THEOREM 3

Let $c_1, c_2, \dots, c_k$ be real numbers. Suppose that the characteristic equation

$$r^k - c_1 r^{k-1} - \dots - c_k = 0$$

has k distinct roots $r_1, r_2, \dots, r_k$. Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$$

if and only if

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \dots + \alpha_k r_k^n$$

for $n = 0, 1, 2, \dots$, where $\alpha_1, \alpha_2, \dots, \alpha_k$ are constants.

We illustrate the use of the theorem with Example 6.

EXAMPLE 6 Find the solution to the recurrence relation

$$a_n = 6a_{n-1} - 11a_{n-2} + 6a_{n-3}$$

with the initial conditions $a_0 = 2$, $a_1 = 5$, and $a_2 = 15$.

Solution: The characteristic polynomial of this recurrence relation is

$$r^3 - 6r^2 + 11r - 6.$$

The characteristic roots are $r = 1$, $r = 2$, and $r = 3$, because $r^3 - 6r^2 + 11r - 6 = (r - 1)(r - 2)(r - 3)$. Hence, the solutions to this recurrence relation are of the form

$$a_n = \alpha_1 \cdot 1^n + \alpha_2 \cdot 2^n + \alpha_3 \cdot 3^n.$$

To find the constants α_1 , α_2 , and α_3 , use the initial conditions. This gives

$$a_0 = 2 = \alpha_1 + \alpha_2 + \alpha_3,$$

$$a_1 = 5 = \alpha_1 + \alpha_2 \cdot 2 + \alpha_3 \cdot 3,$$

$$a_2 = 15 = \alpha_1 + \alpha_2 \cdot 4 + \alpha_3 \cdot 9.$$

When these three simultaneous equations are solved for α_1 , α_2 , and α_3 , we find that $\alpha_1 = 1$, $\alpha_2 = -1$, and $\alpha_3 = 2$. Hence, the unique solution to this recurrence relation and the given initial conditions is the sequence $\{a_n\}$ with

$$a_n = 1 - 2^n + 2 \cdot 3^n.$$

We now state the most general result about linear homogeneous recurrence relations with constant coefficients, allowing the characteristic equation to have multiple roots. The key point is that for each root r of the characteristic equation, the general solution has a summand of the

form $P(n)r^n$, where $P(n)$ is a polynomial of degree $m - 1$, with m the multiplicity of this root. We leave the proof of this result as Exercise 51.

THEOREM 4

Let $c_1, c_2, \dots, c_k$ be real numbers. Suppose that the characteristic equation

$$r^k - c_1 r^{k-1} - \dots - c_k = 0$$

has t distinct roots $r_1, r_2, \dots, r_t$ with multiplicities $m_1, m_2, \dots, m_t$, respectively, so that $m_i \geq 1$ for $i = 1, 2, \dots, t$ and $m_1 + m_2 + \dots + m_t = k$. Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$$

if and only if

$$\begin{aligned} a_n = & (\alpha_{1,0} + \alpha_{1,1}n + \dots + \alpha_{1,m_1-1}n^{m_1-1})r_1^n \\ & + (\alpha_{2,0} + \alpha_{2,1}n + \dots + \alpha_{2,m_2-1}n^{m_2-1})r_2^n \\ & + \dots + (\alpha_{t,0} + \alpha_{t,1}n + \dots + \alpha_{t,m_t-1}n^{m_t-1})r_t^n \end{aligned}$$

for $n = 0, 1, 2, \dots$, where $\alpha_{i,j}$ are constants for $1 \leq i \leq t$ and $0 \leq j \leq m_i - 1$.

Example 7 illustrates how Theorem 4 is used to find the general form of a solution of a linear homogeneous recurrence relation when the characteristic equation has several repeated roots.

EXAMPLE 7

Suppose that the roots of the characteristic equation of a linear homogeneous recurrence relation are 2, 2, 2, 5, 5, and 9 (that is, there are three roots, the root 2 with multiplicity three, the root 5 with multiplicity two, and the root 9 with multiplicity one). What is the form of the general solution?

Solution: By Theorem 4, the general form of the solution is

$$(\alpha_{1,0} + \alpha_{1,1}n + \alpha_{1,2}n^2)2^n + (\alpha_{2,0} + \alpha_{2,1}n)5^n + \alpha_{3,0}9^n.$$

We now illustrate the use of Theorem 4 to solve a linear homogeneous recurrence relation with constant coefficients when the characteristic equation has a root of multiplicity three.

EXAMPLE 8

Find the solution to the recurrence relation

$$a_n = -3a_{n-1} - 3a_{n-2} - a_{n-3}$$

with initial conditions $a_0 = 1$, $a_1 = -2$, and $a_2 = -1$.

Solution: The characteristic equation of this recurrence relation is

$$r^3 + 3r^2 + 3r + 1 = 0.$$

Because $r^3 + 3r^2 + 3r + 1 = (r + 1)^3$, there is a single root $r = -1$ of multiplicity three of the characteristic equation. By Theorem 4 the solutions of this recurrence relation are of the form

$$a_n = \alpha_{1,0}(-1)^n + \alpha_{1,1}n(-1)^n + \alpha_{1,2}n^2(-1)^n.$$

To find the constants $\alpha_{1,0}$, $\alpha_{1,1}$, and $\alpha_{1,2}$, use the initial conditions. This gives

$$\begin{aligned} a_0 &= 1 = \alpha_{1,0}, \\ a_1 &= -2 = -\alpha_{1,0} - \alpha_{1,1} - \alpha_{1,2}, \\ a_2 &= -1 = \alpha_{1,0} + 2\alpha_{1,1} + 4\alpha_{1,2}. \end{aligned}$$

The simultaneous solution of these three equations is $\alpha_{1,0} = 1$, $\alpha_{1,1} = 3$, and $\alpha_{1,2} = -2$. Hence, the unique solution to this recurrence relation and the given initial conditions is the sequence $\{a_n\}$ with

$$a_n = (1 + 3n - 2n^2)(-1)^n.$$

8.2.3 Linear Nonhomogeneous Recurrence Relations with Constant Coefficients

We have seen how to solve linear homogeneous recurrence relations with constant coefficients. Is there a relatively simple technique for solving a linear, but not homogeneous, recurrence relation with constant coefficients, such as $a_n = 3a_{n-1} + 2n$? We will see that the answer is yes for certain families of such recurrence relations.

The recurrence relation $a_n = 3a_{n-1} + 2n$ is an example of a **linear nonhomogeneous recurrence relation with constant coefficients**, that is, a recurrence relation of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} + F(n),$$

where $c_1, c_2, \dots, c_k$ are real numbers and $F(n)$ is a function not identically zero depending only on n . The recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

is called the **associated homogeneous recurrence relation**. It plays an important role in the solution of the nonhomogeneous recurrence relation.

EXAMPLE 9 Each of the recurrence relations $a_n = a_{n-1} + 2^n$, $a_n = a_{n-1} + a_{n-2} + n^2 + n + 1$, $a_n = 3a_{n-1} + n3^n$, and $a_n = a_{n-1} + a_{n-2} + a_{n-3} + n!$ is a linear nonhomogeneous recurrence relation with constant coefficients. The associated linear homogeneous recurrence relations are $a_n = a_{n-1}$, $a_n = a_{n-1} + a_{n-2}$, $a_n = 3a_{n-1}$, and $a_n = a_{n-1} + a_{n-2} + a_{n-3}$, respectively.

The key fact about linear nonhomogeneous recurrence relations with constant coefficients is that every solution is the sum of a particular solution and a solution of the associated linear homogeneous recurrence relation, as Theorem 5 shows.

THEOREM 5

If $\{a_n^{(p)}\}$ is a particular solution of the nonhomogeneous linear recurrence relation with constant coefficients

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} + F(n),$$

then every solution is of the form $\{a_n^{(p)} + a_n^{(h)}\}$, where $\{a_n^{(h)}\}$ is a solution of the associated homogeneous recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

Proof: Because $\{a_n^{(p)}\}$ is a particular solution of the nonhomogeneous recurrence relation, we know that

$$a_n^{(p)} = c_1 a_{n-1}^{(p)} + c_2 a_{n-2}^{(p)} + \cdots + c_k a_{n-k}^{(p)} + F(n).$$

Now suppose that $\{b_n\}$ is a second solution of the nonhomogeneous recurrence relation, so that

$$b_n = c_1 b_{n-1} + c_2 b_{n-2} + \cdots + c_k b_{n-k} + F(n).$$

Subtracting the first of these two equations from the second shows that

$$b_n - a_n^{(p)} = c_1 (b_{n-1} - a_{n-1}^{(p)}) + c_2 (b_{n-2} - a_{n-2}^{(p)}) + \cdots + c_k (b_{n-k} - a_{n-k}^{(p)}).$$

It follows that $\{b_n - a_n^{(p)}\}$ is a solution of the associated homogeneous linear recurrence, say, $\{a_n^{(h)}\}$. Consequently, $b_n = a_n^{(p)} + a_n^{(h)}$ for all n . 

By Theorem 5, we see that the key to solving nonhomogeneous recurrence relations with constant coefficients is finding a particular solution. Then every solution is a sum of this solution and a solution of the associated homogeneous recurrence relation. Although there is no general method for finding such a solution that works for every function $F(n)$, there are techniques that work for certain types of functions $F(n)$, such as polynomials and powers of constants. This is illustrated in Examples 10 and 11.

EXAMPLE 10 Find all solutions of the recurrence relation $a_n = 3a_{n-1} + 2n$. What is the solution with $a_1 = 3$?

Solution: To solve this linear nonhomogeneous recurrence relation with constant coefficients, we need to solve its associated linear homogeneous equation and to find a particular solution for the given nonhomogeneous equation. The associated linear homogeneous equation is $a_n = 3a_{n-1}$. Its solutions are $a_n^{(h)} = \alpha 3^n$, where α is a constant.

We now find a particular solution. Because $F(n) = 2n$ is a polynomial in n of degree one, a reasonable trial solution is a linear function in n , say, $p_n = cn + d$, where c and d are constants. To determine whether there are any solutions of this form, suppose that $p_n = cn + d$ is such a solution. Then the equation $a_n = 3a_{n-1} + 2n$ becomes $cn + d = 3(c(n-1) + d) + 2n$. Simplifying and combining like terms gives $(2 + 2c)n + (2d - 3c) = 0$. It follows that $cn + d$ is a solution if and only if $2 + 2c = 0$ and $2d - 3c = 0$. This shows that $cn + d$ is a solution if and only if $c = -1$ and $d = -3/2$. Consequently, $a_n^{(p)} = -n - 3/2$ is a particular solution.

By Theorem 5 all solutions are of the form

$$a_n = a_n^{(p)} + a_n^{(h)} = -n - \frac{3}{2} + \alpha \cdot 3^n,$$

where α is a constant.

To find the solution with $a_1 = 3$, let $n = 1$ in the formula we obtained for the general solution. We find that $3 = -1 - 3/2 + 3\alpha$, which implies that $\alpha = 11/6$. The solution we seek is $a_n = -n - 3/2 + (11/6)3^n$. 

EXAMPLE 11 Find all solutions of the recurrence relation

**Extra
Examples** 

$$a_n = 5a_{n-1} - 6a_{n-2} + 7^n.$$

Solution: This is a linear nonhomogeneous recurrence relation. The solutions of its associated homogeneous recurrence relation

$$a_n = 5a_{n-1} - 6a_{n-2}$$

are $a_n^{(h)} = \alpha_1 \cdot 3^n + \alpha_2 \cdot 2^n$, where α_1 and α_2 are constants. Because $F(n) = 7^n$, a reasonable trial solution is $a_n^{(p)} = C \cdot 7^n$, where C is a constant. Substituting the terms of this sequence into the recurrence relation implies that $C \cdot 7^n = 5C \cdot 7^{n-1} - 6C \cdot 7^{n-2} + 7^n$. Factoring out 7^{n-2} , this equation becomes $49C = 35C - 6C + 49$, which implies that $20C = 49$, or that $C = 49/20$. Hence, $a_n^{(p)} = (49/20)7^n$ is a particular solution. By Theorem 5, all solutions are of the form

$$a_n = \alpha_1 \cdot 3^n + \alpha_2 \cdot 2^n + (49/20)7^n.$$

In Examples 10 and 11, we made an educated guess that there are solutions of a particular form. In both cases we were able to find particular solutions. This was not an accident. Whenever $F(n)$ is the product of a polynomial in n and the n th power of a constant, we know exactly what form a particular solution has, as stated in Theorem 6. We leave the proof of Theorem 6 as Exercise 52.

THEOREM 6

Suppose that $\{a_n\}$ satisfies the linear nonhomogeneous recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} + F(n),$$

where $c_1, c_2, \dots, c_k$ are real numbers, and

$$F(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0) s^n,$$

where $b_0, b_1, \dots, b_t$ and s are real numbers. When s is not a root of the characteristic equation of the associated linear homogeneous recurrence relation, there is a particular solution of the form

$$(p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n.$$

When s is a root of this characteristic equation and its multiplicity is m , there is a particular solution of the form

$$n^m (p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n.$$

Note that in the case when s is a root of multiplicity m of the characteristic equation of the associated linear homogeneous recurrence relation, the factor n^m ensures that the proposed particular solution will not already be a solution of the associated linear homogeneous recurrence relation. We next provide Example 12 to illustrate the form of a particular solution provided by Theorem 6.

EXAMPLE 12

What form does a particular solution of the linear nonhomogeneous recurrence relation $a_n = 6a_{n-1} - 9a_{n-2} + F(n)$ have when $F(n) = 3^n$, $F(n) = n3^n$, $F(n) = n^2 2^n$, and $F(n) = (n^2 + 1)3^n$?

Solution: The associated linear homogeneous recurrence relation is $a_n = 6a_{n-1} - 9a_{n-2}$. Its characteristic equation, $r^2 - 6r + 9 = (r - 3)^2 = 0$, has a single root, 3, of multiplicity two. To apply Theorem 6, with $F(n)$ of the form $P(n)s^n$, where $P(n)$ is a polynomial and s is a constant, we need to ask whether s is a root of this characteristic equation.

Because $s = 3$ is a root with multiplicity $m = 2$ but $s = 2$ is not a root, Theorem 6 tells us that a particular solution has the form $p_0 n^2 3^n$ if $F(n) = 3^n$, the form $n^2(p_1 n + p_0)3^n$ if $F(n) =$

$n3^n$, the form $(p_2n^2 + p_1n + p_0)2^n$ if $F(n) = n^22^n$, and the form $n^2(p_2n^2 + p_1n + p_0)3^n$ if $F(n) = (n^2 + 1)3^n$. 

Care must be taken when $s = 1$ when solving recurrence relations of the type covered by Theorem 6. In particular, to apply this theorem with $F(n) = b_in_t + b_{t-1}n_{t-1} + \cdots + b_1n + b_0$, the parameter s takes the value $s = 1$ (even though the term 1^n does not explicitly appear). By the theorem, the form of the solution then depends on whether 1 is a root of the characteristic equation of the associated linear homogeneous recurrence relation. This is illustrated in Example 13, which shows how Theorem 6 can be used to find a formula for the sum of the first n positive integers.

EXAMPLE 13 Let a_n be the sum of the first n positive integers, so that

$$a_n = \sum_{k=1}^n k.$$

Note that a_n satisfies the linear nonhomogeneous recurrence relation

$$a_n = a_{n-1} + n.$$

(To obtain a_n , the sum of the first n positive integers, from a_{n-1} , the sum of the first $n - 1$ positive integers, we add n .) Note that the initial condition is $a_1 = 1$.

The associated linear homogeneous recurrence relation for a_n is

$$a_n = a_{n-1}.$$

The solutions of this homogeneous recurrence relation are given by $a_n^{(h)} = c(1)^n = c$, where c is a constant. To find all solutions of $a_n = a_{n-1} + n$, we need find only a single particular solution. By Theorem 6, because $F(n) = n = n \cdot (1)^n$ and $s = 1$ is a root of degree one of the characteristic equation of the associated linear homogeneous recurrence relation, there is a particular solution of the form $n(p_1n + p_0) = p_1n^2 + p_0n$.

Inserting this into the recurrence relation gives $p_1n^2 + p_0n = p_1(n-1)^2 + p_0(n-1) + n$. Simplifying, we see that $n(2p_1 - 1) + (p_0 - p_1) = 0$, which means that $2p_1 - 1 = 0$ and $p_0 - p_1 = 0$, so $p_0 = p_1 = 1/2$. Hence,

$$a_n^{(p)} = \frac{n^2}{2} + \frac{n}{2} = \frac{n(n+1)}{2}$$

is a particular solution. Hence, all solutions of the original recurrence relation $a_n = a_{n-1} + n$ are given by $a_n = a_n^{(h)} + a_n^{(p)} = c + n(n+1)/2$. Because $a_1 = 1$, we have $1 = a_1 = c + 1 \cdot 2/2 = c + 1$, so $c = 0$. It follows that $a_n = n(n+1)/2$. (This is the same formula given in Table 2 in Section 2.4 and derived previously.) 

Exercises

- Determine which of these are linear homogeneous recurrence relations with constant coefficients. Also, find the degree of those that are.
 - $a_n = 3a_{n-1} + 4a_{n-2} + 5a_{n-3}$
 - $a_n = 2na_{n-1} + a_{n-2}$
 - $a_n = a_{n-1} + a_{n-4}$
 - $a_n = a_{n-1} + 2$
 - $a_n = a_{n-1}^2 + a_{n-2}$
 - $a_n = a_{n-1} + n$
- Determine which of these are linear homogeneous recurrence relations with constant coefficients. Also, find the degree of those that are.
 - $a_n = 3a_{n-2}$
 - $a_n = 3$
 - $a_n = a_{n-1}^2/n$
 - $a_n = a_{n-1} + 2a_{n-3}$
 - $a_n = a_{n-1} + a_{n-2} + n + 3$
 - $a_n = 4a_{n-2} + 5a_{n-4} + 9a_{n-7}$

3. Solve these recurrence relations together with the initial conditions given.

- a) $a_n = 2a_{n-1}$ for $n \geq 1$, $a_0 = 3$
- b) $a_n = a_{n-1}$ for $n \geq 1$, $a_0 = 2$
- c) $a_n = 5a_{n-1} - 6a_{n-2}$ for $n \geq 2$, $a_0 = 1$, $a_1 = 0$
- d) $a_n = 4a_{n-1} - 4a_{n-2}$ for $n \geq 2$, $a_0 = 6$, $a_1 = 8$
- e) $a_n = -4a_{n-1} - 4a_{n-2}$ for $n \geq 2$, $a_0 = 0$, $a_1 = 1$
- f) $a_n = 4a_{n-2}$ for $n \geq 2$, $a_0 = 0$, $a_1 = 4$
- g) $a_n = a_{n-2}/4$ for $n \geq 2$, $a_0 = 1$, $a_1 = 0$

4. Solve these recurrence relations together with the initial conditions given.

- a) $a_n = a_{n-1} + 6a_{n-2}$ for $n \geq 2$, $a_0 = 3$, $a_1 = 6$
- b) $a_n = 7a_{n-1} - 10a_{n-2}$ for $n \geq 2$, $a_0 = 2$, $a_1 = 1$
- c) $a_n = 6a_{n-1} - 8a_{n-2}$ for $n \geq 2$, $a_0 = 4$, $a_1 = 10$
- d) $a_n = 2a_{n-1} - a_{n-2}$ for $n \geq 2$, $a_0 = 4$, $a_1 = 1$
- e) $a_n = a_{n-2}$ for $n \geq 2$, $a_0 = 5$, $a_1 = -1$
- f) $a_n = -6a_{n-1} - 9a_{n-2}$ for $n \geq 2$, $a_0 = 3$, $a_1 = -3$
- g) $a_{n+2} = -4a_{n+1} + 5a_n$ for $n \geq 0$, $a_0 = 2$, $a_1 = 8$

5. How many different messages can be transmitted in n microseconds using the two signals described in Exercise 19 in Section 8.1?

6. How many different messages can be transmitted in n microseconds using three different signals if one signal requires 1 microsecond for transmittal, the other two signals require 2 microseconds each for transmittal, and a signal in a message is followed immediately by the next signal?

7. In how many ways can a $2 \times n$ rectangular checkerboard be tiled using 1×2 and 2×2 pieces?

8. A model for the number of lobsters caught per year is based on the assumption that the number of lobsters caught in a year is the average of the number caught in the two previous years.

- a) Find a recurrence relation for $\{L_n\}$, where L_n is the number of lobsters caught in year n , under the assumption for this model.
- b) Find L_n if 100,000 lobsters were caught in year 1 and 300,000 were caught in year 2.

9. A deposit of \$100,000 is made to an investment fund at the beginning of a year. On the last day of each year two dividends are awarded. The first dividend is 20% of the amount in the account during that year. The second dividend is 45% of the amount in the account in the previous year.

- a) Find a recurrence relation for $\{P_n\}$, where P_n is the amount in the account at the end of n years if no money is ever withdrawn.
- b) How much is in the account after n years if no money has been withdrawn?

- *10. Prove Theorem 2.

11. The **Lucas numbers** satisfy the recurrence relation

Links $L_n = L_{n-1} + L_{n-2},$

and the initial conditions $L_0 = 2$ and $L_1 = 1$.

- a) Show that $L_n = f_{n-1} + f_{n+1}$ for $n = 2, 3, \dots$, where f_n is the n th Fibonacci number.
- b) Find an explicit formula for the Lucas numbers.

12. Find the solution to $a_n = 2a_{n-1} + a_{n-2} - 2a_{n-3}$ for $n = 3, 4, 5, \dots$, with $a_0 = 3$, $a_1 = 6$, and $a_2 = 0$.

13. Find the solution to $a_n = 7a_{n-2} + 6a_{n-3}$ with $a_0 = 9$, $a_1 = 10$, and $a_2 = 32$.

14. Find the solution to $a_n = 5a_{n-2} - 4a_{n-4}$ with $a_0 = 3$, $a_1 = 2$, $a_2 = 6$, and $a_3 = 8$.

15. Find the solution to $a_n = 2a_{n-1} + 5a_{n-2} - 6a_{n-3}$ with $a_0 = 7$, $a_1 = -4$, and $a_2 = 8$.

- *16. Prove Theorem 3.

17. Prove this identity relating the Fibonacci numbers and the binomial coefficients:

$$f_{n+1} = C(n, 0) + C(n-1, 1) + \dots + C(n-k, k),$$

where n is a positive integer and $k = \lfloor n/2 \rfloor$. [Hint: Let $a_n = C(n, 0) + C(n-1, 1) + \dots + C(n-k, k)$. Show that the sequence $\{a_n\}$ satisfies the same recurrence relation and initial conditions satisfied by the sequence of Fibonacci numbers.]

18. Solve the recurrence relation $a_n = 6a_{n-1} - 12a_{n-2} + 8a_{n-3}$ with $a_0 = -5$, $a_1 = 4$, and $a_2 = 88$.

19. Solve the recurrence relation $a_n = -3a_{n-1} - 3a_{n-2} - a_{n-3}$ with $a_0 = 5$, $a_1 = -9$, and $a_2 = 15$.

20. Find the general form of the solutions of the recurrence relation $a_n = 8a_{n-2} - 16a_{n-4}$.

21. What is the general form of the solutions of a linear homogeneous recurrence relation if its characteristic equation has roots 1, 1, 1, 1, -2, -2, -2, 3, 3, -4?

22. What is the general form of the solutions of a linear homogeneous recurrence relation if its characteristic equation has the roots -1, -1, -1, 2, 2, 5, 5, 7?

23. Consider the nonhomogeneous linear recurrence relation $a_n = 3a_{n-1} + 2^n$.

- a) Show that $a_n = -2^{n+1}$ is a solution of this recurrence relation.

- b) Use Theorem 5 to find all solutions of this recurrence relation.

- c) Find the solution with $a_0 = 1$.

24. Consider the nonhomogeneous linear recurrence relation $a_n = 2a_{n-1} + 2^n$.

- a) Show that $a_n = n2^n$ is a solution of this recurrence relation.

- b) Use Theorem 5 to find all solutions of this recurrence relation.

- c) Find the solution with $a_0 = 2$.

25. a) Determine values of the constants A and B such that $a_n = An + B$ is a solution of recurrence relation $a_n = 2a_{n-1} + n + 5$.

- b) Use Theorem 5 to find all solutions of this recurrence relation.

- c) Find the solution of this recurrence relation with $a_0 = 4$.

26. What is the general form of the particular solution guaranteed to exist by Theorem 6 of the linear nonhomogeneous recurrence relation $a_n = 6a_{n-1} - 12a_{n-2} + 8a_{n-3} + F(n)$ if
- $F(n) = n^2?$
 - $F(n) = 2^n?$
 - $F(n) = n2^n?$
 - $F(n) = (-2)^n?$
 - $F(n) = n^22^n?$
 - $F(n) = n^3(-2)^n?$
 - $F(n) = 3?$
27. What is the general form of the particular solution guaranteed to exist by Theorem 6 of the linear nonhomogeneous recurrence relation $a_n = 8a_{n-2} - 16a_{n-4} + F(n)$ if
- $F(n) = n^3?$
 - $F(n) = (-2)^n?$
 - $F(n) = n2^n?$
 - $F(n) = n^24^n?$
 - $F(n) = (n^2 - 2)(-2)^n?$
 - $F(n) = n^42^n?$
 - $F(n) = 2?$
28. a) Find all solutions of the recurrence relation $a_n = 2a_{n-1} + 2n^2$.
b) Find the solution of the recurrence relation in part (a) with initial condition $a_1 = 4$.
29. a) Find all solutions of the recurrence relation $a_n = 2a_{n-1} + 3^n$.
b) Find the solution of the recurrence relation in part (a) with initial condition $a_1 = 5$.
30. a) Find all solutions of the recurrence relation $a_n = -5a_{n-1} - 6a_{n-2} + 42 \cdot 4^n$.
b) Find the solution of this recurrence relation with $a_1 = 56$ and $a_2 = 278$.
31. Find all solutions of the recurrence relation $a_n = 5a_{n-1} - 6a_{n-2} + 2^n + 3n$. [Hint: Look for a particular solution of the form $qn2^n + p_1n + p_2$, where q , p_1 , and p_2 are constants.]
32. Find the solution of the recurrence relation $a_n = 2a_{n-1} + 3 \cdot 2^n$.
33. Find all solutions of the recurrence relation $a_n = 4a_{n-1} - 4a_{n-2} + (n+1)2^n$.
34. Find all solutions of the recurrence relation $a_n = 7a_{n-1} - 16a_{n-2} + 12a_{n-3} + n4^n$ with $a_0 = -2$, $a_1 = 0$, and $a_2 = 5$.
35. Find the solution of the recurrence relation $a_n = 4a_{n-1} - 3a_{n-2} + 2^n + n + 3$ with $a_0 = 1$ and $a_1 = 4$.
36. Let a_n be the sum of the first n perfect squares, that is, $a_n = \sum_{k=1}^n k^2$. Show that the sequence $\{a_n\}$ satisfies the linear nonhomogeneous recurrence relation $a_n = a_{n-1} + n^2$ and the initial condition $a_1 = 1$. Use Theorem 6 to determine a formula for a_n by solving this recurrence relation.
37. Let a_n be the sum of the first n triangular numbers, that is, $a_n = \sum_{k=1}^n t_k$, where $t_k = k(k+1)/2$. Show that $\{a_n\}$ satisfies the linear nonhomogeneous recurrence relation $a_n = a_{n-1} + n(n+1)/2$ and the initial condition $a_1 = 1$. Use Theorem 6 to determine a formula for a_n by solving this recurrence relation.
38. a) Find the characteristic roots of the linear homogeneous recurrence relation $a_n = 2a_{n-1} - 2a_{n-2}$. [Note: These are complex numbers.]

- b) Find the solution of the recurrence relation in part (a) with $a_0 = 1$ and $a_1 = 2$.

*39. a) Find the characteristic roots of the linear homogeneous recurrence relation $a_n = a_{n-4}$. [Note: These include complex numbers.]

- b) Find the solution of the recurrence relation in part (a) with $a_0 = 1$, $a_1 = 0$, $a_2 = -1$, and $a_3 = 1$.

*40. Solve the simultaneous recurrence relations

$$a_n = 3a_{n-1} + 2b_{n-1}$$

$$b_n = a_{n-1} + 2b_{n-1}$$

with $a_0 = 1$ and $b_0 = 2$.

*41. a) Use the formula found in Example 4 for f_n , the n th Fibonacci number, to show that f_n is the integer closest to

$$\frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n.$$

- b) Determine for which n f_n is greater than

$$\frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n$$

and for which n f_n is less than

$$\frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n.$$

42. Show that if $a_n = a_{n-1} + a_{n-2}$, $a_0 = s$ and $a_1 = t$, where s and t are constants, then $a_n = sf_{n-1} + tf_n$ for all positive integers n .

43. Express the solution of the linear nonhomogeneous recurrence relation $a_n = a_{n-1} + a_{n-2} + 1$ for $n \geq 2$ where $a_0 = 0$ and $a_1 = 1$ in terms of the Fibonacci numbers. [Hint: Let $b_n = a_n + 1$ and apply Exercise 42 to the sequence b_n .]

*44. (Linear algebra required) Let $\mathbf{A}_n$ be the $n \times n$ matrix with 2s on its main diagonal, 1s in all positions next to a diagonal element, and 0s everywhere else. Find a recurrence relation for d_n , the determinant of $\mathbf{A}_n$. Solve this recurrence relation to find a formula for d_n .

45. Suppose that each pair of a genetically engineered species of rabbits left on an island produces two new pairs of rabbits at the age of 1 month and six new pairs of rabbits at the age of 2 months and every month afterward. None of the rabbits ever die or leave the island.

- Find a recurrence relation for the number of pairs of rabbits on the island n months after one newborn pair is left on the island.
- By solving the recurrence relation in (a) determine the number of pairs of rabbits on the island n months after one pair is left on the island.

46. Suppose that there are two goats on an island initially. The number of goats on the island doubles every year by natural reproduction, and some goats are either added or removed each year.

- a) Construct a recurrence relation for the number of goats on the island at the start of the n th year, assuming that during each year an extra 100 goats are put on the island.
 - b) Solve the recurrence relation from part (a) to find the number of goats on the island at the start of the n th year.
 - c) Construct a recurrence relation for the number of goats on the island at the start of the n th year, assuming that n goats are removed during the n th year for each $n \geq 3$.
 - d) Solve the recurrence relation in part (c) for the number of goats on the island at the start of the n th year.
47. A new employee at an exciting new software company starts with a salary of \$50,000 and is promised that at the end of each year her salary will be double her salary of the previous year, with an extra increment of \$10,000 for each year she has been with the company.
- a) Construct a recurrence relation for her salary for her n th year of employment.
 - b) Solve this recurrence relation to find her salary for her n th year of employment.

Some linear recurrence relations that do not have constant coefficients can be systematically solved. This is the case for recurrence relations of the form $f(n)a_n = g(n)a_{n-1} + h(n)$. Exercises 48–50 illustrate this.

- *48. a) Show that the recurrence relation

$$f(n)a_n = g(n)a_{n-1} + h(n),$$

for $n \geq 1$, and with $a_0 = C$, can be reduced to a recurrence relation of the form

$$b_n = b_{n-1} + Q(n)h(n),$$

where $b_n = g(n+1)Q(n+1)a_n$, with

$$Q(n) = (f(1)f(2) \cdots f(n-1))/(g(1)g(2) \cdots g(n)).$$

- b) Use part (a) to solve the original recurrence relation to obtain

$$a_n = \frac{C + \sum_{i=1}^n Q(i)h(i)}{g(n+1)Q(n+1)}.$$

- *49. Use Exercise 48 to solve the recurrence relation $(n+1)a_n = (n+3)a_{n-1} + n$, for $n \geq 1$, with $a_0 = 1$.
50. It can be shown that C_n , the average number of comparisons made by the quick sort algorithm (described in preamble to Exercise 50 in Section 5.4), when sorting n elements in random order, satisfies the recurrence relation

$$C_n = n + 1 + \frac{2}{n} \sum_{k=0}^{n-1} C_k$$

for $n = 1, 2, \dots$, with initial condition $C_0 = 0$.

- a) Show that $\{C_n\}$ also satisfies the recurrence relation $nC_n = (n+1)C_{n-1} + 2n$ for $n = 1, 2, \dots$.
- b) Use Exercise 48 to solve the recurrence relation in part (a) to find an explicit formula for C_n .

**51. Prove Theorem 4.

**52. Prove Theorem 6.

53. Solve the recurrence relation $T(n) = nT^2(n/2)$ with initial condition $T(1) = 6$ when $n = 2^k$ for some integer k . [Hint: Let $n = 2^k$ and then make the substitution $a_k = \log T(2^k)$ to obtain a linear nonhomogeneous recurrence relation.]

8.3 Divide-and-Conquer Algorithms and Recurrence Relations

8.3.1 Introduction



Many recursive algorithms take a problem with a given input and divide it into one or more smaller problems. This reduction is successively applied until the solutions of the smaller problems can be found quickly. For instance, we perform a binary search by reducing the search for an element in a list to the search for this element in a list half as long. We successively apply this reduction until one element is left. When we sort a list of integers using the merge sort, we split the list into two halves of equal size and sort each half separately. We then merge the two sorted halves. Another example of this type of recursive algorithm is a procedure for multiplying integers that reduces the problem of the multiplication of two integers to three multiplications of pairs of integers with half as many bits. This reduction is successively applied until integers with one bit are obtained. These procedures follow an important algorithmic paradigm known as **divide-and-conquer**, and are called **divide-and-conquer algorithms**, because they *divide* a problem into one or more instances of the same problem of smaller size and they *conquer* the problem by using the solutions of the smaller problems to find a solution of the original problem, perhaps with some additional work.

In this section we will show how recurrence relations can be used to analyze the computational complexity of divide-and-conquer algorithms. We will use these recurrence relations

“Divide et impera”
(translation: “Divide
and conquer”) —Julius
Caesar

to estimate the number of operations used by many different divide-and-conquer algorithms, including several that we introduce in this section.

8.3.2 Divide-and-Conquer Recurrence Relations

Suppose that a recursive algorithm divides a problem of size n into a subproblems, where each subproblem is of size n/b (for simplicity, assume that n is a multiple of b ; in reality, the smaller problems are often of size equal to the nearest integers either less than or equal to, or greater than or equal to, n/b). Also, suppose that a total of $g(n)$ extra operations are required in the conquer step of the algorithm to combine the solutions of the subproblems into a solution of the original problem. Then, if $f(n)$ represents the number of operations required to solve the problem of size n , it follows that f satisfies the recurrence relation

$$f(n) = af(n/b) + g(n).$$

This is called a **divide-and-conquer recurrence relation**.

We will first set up the divide-and-conquer recurrence relations that can be used to study the complexity of some important algorithms. Then we will show how to use these divide-and-conquer recurrence relations to estimate the complexity of these algorithms.

EXAMPLE 1 Binary Search We introduced a binary search algorithm in Section 3.1. This binary search algorithm reduces the search for an element in a search sequence of size n to the binary search for this element in a search sequence of size $n/2$, when n is even. (Hence, the problem of size n has been reduced to *one* problem of size $n/2$.) Two comparisons are needed to implement this reduction (one to determine which half of the list to use and the other to determine whether any terms of the list remain). Hence, if $f(n)$ is the number of comparisons required to search for an element in a search sequence of size n , then

*Extra
Examples* ➤

$$f(n) = f(n/2) + 2$$

when n is even. ◀

EXAMPLE 2 Finding the Maximum and Minimum of a Sequence Consider the following algorithm for locating the maximum and minimum elements of a sequence $a_1, a_2, \dots, a_n$. If $n = 1$, then a_1 is the maximum and the minimum. If $n > 1$, split the sequence into two sequences, either where both have the same number of elements or where one of the sequences has one more element than the other. The problem is reduced to finding the maximum and minimum of each of the two smaller sequences. The solution to the original problem results from the comparison of the separate maxima and minima of the two smaller sequences to obtain the overall maximum and minimum.

Let $f(n)$ be the total number of comparisons needed to find the maximum and minimum elements of the sequence with n elements. We have shown that a problem of size n can be reduced into two problems of size $n/2$, when n is even, using two comparisons, one to compare the maxima of the two sequences and the other to compare the minima of the two sequences. This gives the recurrence relation

$$f(n) = 2f(n/2) + 2$$

when n is even. ◀

EXAMPLE 3 Merge Sort The merge sort algorithm (introduced in Section 5.4) splits a list to be sorted with n items, where n is even, into two lists with $n/2$ elements each, and uses fewer than n

comparisons to merge the two sorted lists of $n/2$ items each into one sorted list. Consequently, the number of comparisons used by the merge sort to sort a list of n elements is less than $M(n)$, where the function $M(n)$ satisfies the divide-and-conquer recurrence relation

$$M(n) = 2M(n/2) + n.$$

EXAMPLE 4 Fast Multiplication of Integers Surprisingly, there are more efficient algorithms than the conventional algorithm (described in Section 4.2) for multiplying integers. One of these algorithms, which uses a divide-and-conquer technique, will be described here. This fast multiplication algorithm proceeds by splitting each of two $2n$ -bit integers into two blocks, each with n bits. Then, the original multiplication is reduced from the multiplication of two $2n$ -bit integers to three multiplications of n -bit integers, plus shifts and additions.

Links

Suppose that a and b are integers with binary expansions of length $2n$ (add initial bits of zero in these expansions if necessary to make them the same length). Let

$$a = (a_{2n-1}a_{2n-2} \cdots a_1a_0)_2 \quad \text{and} \quad b = (b_{2n-1}b_{2n-2} \cdots b_1b_0)_2.$$

Let

$$a = 2^n A_1 + A_0, \quad b = 2^n B_1 + B_0,$$

where

$$\begin{aligned} A_1 &= (a_{2n-1} \cdots a_{n+1}a_n)_2, & A_0 &= (a_{n-1} \cdots a_1a_0)_2, \\ B_1 &= (b_{2n-1} \cdots b_{n+1}b_n)_2, & B_0 &= (b_{n-1} \cdots b_1b_0)_2. \end{aligned}$$

The algorithm for fast multiplication of integers is based on the fact that ab can be rewritten as



$$ab = (2^{2n} + 2^n)A_1B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0B_0.$$

The important fact about this identity is that it shows that the multiplication of two $2n$ -bit integers can be carried out using three multiplications of n -bit integers, together with additions, subtractions, and shifts. This shows that if $f(n)$ is the total number of bit operations needed to multiply two n -bit integers, then

$$f(2n) = 3f(n) + Cn.$$

The reasoning behind this equation is as follows. The three multiplications of n -bit integers are carried out using $3f(n)$ -bit operations. Each of the additions, subtractions, and shifts uses a constant multiple of n -bit operations, and Cn represents the total number of bit operations used by these operations.

EXAMPLE 5 Fast Matrix Multiplication In Example 7 of Section 3.3 we showed that multiplying two $n \times n$ matrices using the definition of matrix multiplication required n^3 multiplications and $n^2(n - 1)$ additions. Consequently, computing the product of two $n \times n$ matrices in this way requires $O(n^3)$ operations (multiplications and additions). Surprisingly, there are more efficient divide-and-conquer algorithms for multiplying two $n \times n$ matrices. Such an algorithm, invented by Volker Strassen in 1969, reduces the multiplication of two $n \times n$ matrices, when n is even, to seven multiplications of two $(n/2) \times (n/2)$ matrices and 15 additions of $(n/2) \times (n/2)$ matrices.

Links

(See [CoLeRiSt09] for the details of this algorithm.) Hence, if $f(n)$ is the number of operations (multiplications and additions) used, it follows that

$$f(n) = 7f(n/2) + 15n^2/4$$

when n is even. ◀

As Examples 1–5 show, recurrence relations of the form $f(n) = af(n/b) + g(n)$ arise in many different situations. It is possible to derive estimates of the size of functions that satisfy such recurrence relations. Suppose that f satisfies this recurrence relation whenever n is divisible by b . Let $n = b^k$, where k is a positive integer. Then

$$\begin{aligned} f(n) &= af(n/b) + g(n) \\ &= a^2f(n/b^2) + ag(n/b) + g(n) \\ &= a^3f(n/b^3) + a^2g(n/b^2) + ag(n/b) + g(n) \\ &\vdots \\ &= a^kf(n/b^k) + \sum_{j=0}^{k-1} a^jg(n/b^j). \end{aligned}$$

Because $n/b^k = 1$, it follows that

$$f(n) = a^kf(1) + \sum_{j=0}^{k-1} a^jg(n/b^j).$$

We can use this equation for $f(n)$ to estimate the size of functions that satisfy divide-and-conquer relations.

THEOREM 1

Let f be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + c$$

whenever n is divisible by b , where $a \geq 1$, b is an integer greater than 1, and c is a positive real number. Then

$$f(n) \text{ is } \begin{cases} O(n^{\log_b a}) & \text{if } a > 1, \\ O(\log n) & \text{if } a = 1. \end{cases}$$

Furthermore, when $n = b^k$ and $a \neq 1$, where k is a positive integer,

$$f(n) = C_1n^{\log_b a} + C_2,$$

where $C_1 = f(1) + c/(a - 1)$ and $C_2 = -c/(a - 1)$.



Proof: First let $n = b^k$. From the expression for $f(n)$ obtained in the discussion preceding the theorem, with $g(n) = c$, we have

$$f(n) = a^kf(1) + \sum_{j=0}^{k-1} a^jc = a^kf(1) + c \sum_{j=0}^{k-1} a^j.$$

When $a = 1$ we have

$$f(n) = f(1) + ck.$$

Because $n = b^k$, we have $k = \log_b n$. Hence,

$$f(n) = f(1) + c \log_b n.$$

When n is not a power of b , we have $b^k < n < b^{k+1}$, for a positive integer k . Because f is increasing, it follows that $f(n) \leq f(b^{k+1}) = f(1) + c(k+1) = (f(1) + c) + ck \leq (f(1) + c) + c \log_b n$. Therefore, in both cases, $f(n)$ is $O(\log n)$ when $a = 1$.

Now suppose that $a > 1$. First assume that $n = b^k$, where k is a positive integer. From the formula for the sum of terms of a geometric progression (Theorem 1 in Section 2.4), it follows that

$$\begin{aligned} f(n) &= a^k f(1) + c(a^k - 1)/(a - 1) \\ &= a^k [f(1) + c/(a - 1)] - c/(a - 1) \\ &= C_1 n^{\log_b a} + C_2, \end{aligned}$$

because $a^k = a^{\log_b n} = n^{\log_b a}$ (see Exercise 4 in Appendix 2), where $C_1 = f(1) + c/(a - 1)$ and $C_2 = -c/(a - 1)$.

Now suppose that n is not a power of b . Then $b^k < n < b^{k+1}$, where k is a nonnegative integer. Because f is increasing,

$$\begin{aligned} f(n) &\leq f(b^{k+1}) = C_1 a^{k+1} + C_2 \\ &\leq (C_1 a) a^{\log_b n} + C_2 \\ &= (C_1 a) n^{\log_b a} + C_2, \end{aligned}$$

because $k \leq \log_b n < k + 1$.

Hence, we have $f(n)$ is $O(n^{\log_b a})$. ◀

Examples 6–9 illustrate how Theorem 1 is used.

EXAMPLE 6 Let $f(n) = 5f(n/2) + 3$ and $f(1) = 7$. Find $f(2^k)$, where k is a positive integer. Also, estimate $f(n)$ if f is an increasing function.

Extra Examples ▶

Solution: From the proof of Theorem 1, with $a = 5$, $b = 2$, and $c = 3$, we see that if $n = 2^k$, then

$$\begin{aligned} f(n) &= a^k [f(1) + c/(a - 1)] + [-c/(a - 1)] \\ &= 5^k [7 + (3/4)] - 3/4 \\ &= 5^k (31/4) - 3/4. \end{aligned}$$

Also, if $f(n)$ is increasing, Theorem 1 shows that $f(n)$ is $O(n^{\log_b a}) = O(n^{\log 5})$. ◀

We can use Theorem 1 to estimate the computational complexity of the binary search algorithm and the algorithm given in Example 2 for locating the minimum and maximum of a sequence.

EXAMPLE 7 Give a big- O estimate for the number of comparisons used by a binary search.

Solution: In Example 1 it was shown that $f(n) = f(n/2) + 2$ when n is even, where f is the number of comparisons required to perform a binary search on a sequence of size n . Hence, from Theorem 1, it follows that $f(n)$ is $O(\log n)$. 

EXAMPLE 8 Give a big- O estimate for the number of comparisons used to locate the maximum and minimum elements in a sequence using the algorithm given in Example 2.

Solution: In Example 2 we showed that $f(n) = 2f(n/2) + 2$, when n is even, where f is the number of comparisons needed by this algorithm. Hence, from Theorem 1, it follows that $f(n)$ is $O(n^{\log 2}) = O(n)$. 

We now state a more general, and more complicated, theorem, which has Theorem 1 as a special case. This theorem (or more powerful versions, including big-Theta estimates) is sometimes known as the master theorem because it is useful in analyzing the complexity of many important divide-and-conquer algorithms.

THEOREM 2 MASTER THEOREM Let f be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + cn^d$$

whenever $n = b^k$, where k is a positive integer, $a \geq 1$, b is an integer greater than 1, and c and d are real numbers with c positive and d nonnegative. Then

$$f(n) \text{ is } \begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

The proof of Theorem 2 is left for the reader as Exercises 29–33.

EXAMPLE 9 Complexity of Merge Sort In Example 3 we explained that the number of comparisons used by the merge sort to sort a list of n elements is less than $M(n)$, where $M(n) = 2M(n/2) + n$. By the master theorem (Theorem 2) we find that $M(n)$ is $O(n \log n)$, which agrees with the estimate found in Section 5.4. 

EXAMPLE 10 Give a big- O estimate for the number of bit operations needed to multiply two n -bit integers using the fast multiplication algorithm described in Example 4.

Solution: Example 4 shows that $f(n) = 3f(n/2) + Cn$, when n is even, where $f(n)$ is the number of bit operations required to multiply two n -bit integers using the fast multiplication algorithm. Hence, from the master theorem (Theorem 2), it follows that $f(n)$ is $O(n^{\log 3})$. Note that $\log 3 \sim 1.6$. Because the conventional algorithm for multiplication uses $O(n^2)$ bit operations, the fast multiplication algorithm is a substantial improvement over the conventional algorithm in

terms of time complexity for sufficiently large integers, including large integers that occur in practical applications. 

EXAMPLE 11 Give a big- O estimate for the number of multiplications and additions required to multiply two $n \times n$ matrices using the matrix multiplication algorithm referred to in Example 5.

Solution: Let $f(n)$ denote the number of additions and multiplications used by the algorithm mentioned in Example 5 to multiply two $n \times n$ matrices. We have $f(n) = 7f(n/2) + 15n^2/4$, when n is even. Hence, from the master theorem (Theorem 2), it follows that $f(n)$ is $O(n^{\log 7})$. Note that $\log 7 \sim 2.8$. Because the conventional algorithm for multiplying two $n \times n$ matrices uses $O(n^3)$ additions and multiplications, it follows that for sufficiently large integers n , including those that occur in many practical applications, this algorithm is substantially more efficient in time complexity than the conventional algorithm. 

THE CLOSEST-PAIR PROBLEM We conclude this section by introducing a divide-and-conquer algorithm from computational geometry, the part of discrete mathematics devoted to algorithms that solve geometric problems.

EXAMPLE 12 The Closest-Pair Problem Consider the problem of determining the closest pair of points in a set of n points $(x_1, y_1), \dots, (x_n, y_n)$ in the plane, where the distance between two points (x_i, y_i) and (x_j, y_j) is the usual Euclidean distance $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. This problem arises in many applications such as determining the closest pair of airplanes in the air space at a particular altitude being managed by an air traffic controller. How can this closest pair of points be found in an efficient way?

 **Links**

It took researchers more than 10 years to find an algorithm with $O(n \log n)$ complexity that locates the closest pair of points among n points.

Solution: To solve this problem we can first determine the distance between every pair of points and then find the smallest of these distances. However, this approach requires $O(n^2)$ computations of distances and comparisons because there are $C(n, 2) = n(n-1)/2$ pairs of points. Surprisingly, there is an elegant divide-and-conquer algorithm that can solve the closest-pair problem for n points using $O(n \log n)$ computations of distances and comparisons. The algorithm we describe here is due to Michael Samos (see [PrSa85]).

For simplicity, we assume that $n = 2^k$, where k is a positive integer. (We avoid some technical considerations that are needed when n is not a power of 2.) When $n = 2$, we have only one pair of points; the distance between these two points is the minimum distance. At the start of the algorithm we use the merge sort twice, once to sort the points in order of increasing x coordinates, and once to sort the points in order of increasing y coordinates. Each of these sorts requires $O(n \log n)$ operations. We will use these sorted lists in each recursive step.

The recursive part of the algorithm divides the problem into two subproblems, each involving half as many points. Using the sorted list of the points by their x coordinates, we construct a vertical line ℓ dividing the n points into two parts, a left part and a right part of equal size, each containing $n/2$ points, as shown in Figure 1. (If any points fall on the dividing line ℓ , we divide them among the two parts if necessary.) At subsequent steps of the recursion we need not sort on x coordinates again, because we can select the corresponding sorted subset of all the points. This selection is a task that can be done with $O(n)$ comparisons.

There are three possibilities concerning the positions of the closest points: (1) they are both in the left region L , (2) they are both in the right region R , or (3) one point is in the left region and the other is in the right region. Apply the algorithm recursively to compute d_L and d_R , where d_L is the minimum distance between points in the left region and d_R is the minimum distance between points in the right region. Let $d = \min(d_L, d_R)$. To successfully divide the problem of finding the closest two points in the original set into the two problems of finding the

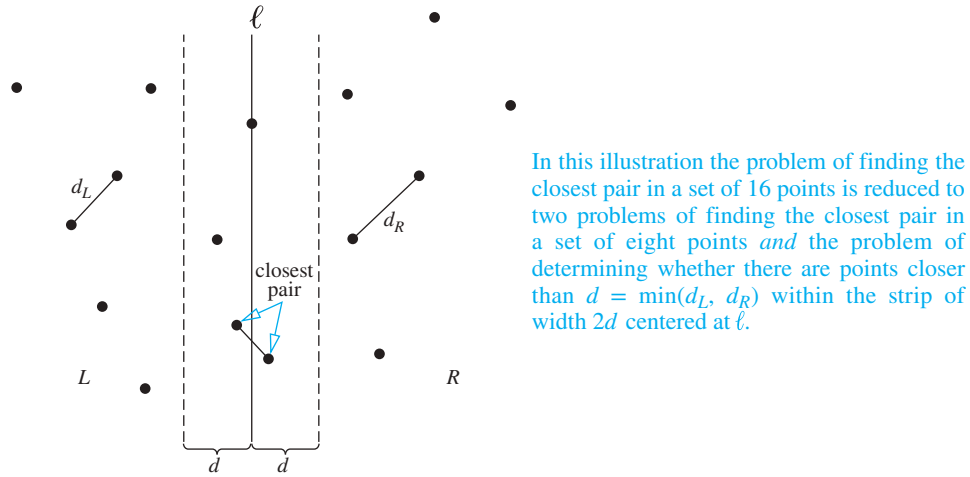


FIGURE 1 The recursive step of the algorithm for solving the closest-pair problem.

shortest distances between points in the two regions separately, we have to handle the conquer part of the algorithm, which requires that we consider the case where the closest points lie in different regions, that is, one point is in L and the other in R . Because there is a pair of points at distance d where both points lie in R or both points lie in L , for the closest points to lie in different regions requires that they must be a distance less than d apart.

For a point in the left region and a point in the right region to lie at a distance less than d apart, these points must lie in the vertical strip of width $2d$ that has the line ℓ as its center. (Otherwise, the distance between these points is greater than the difference in their x coordinates, which exceeds d .) To examine the points within this strip, we sort the points so that they are listed in order of increasing y coordinates, using the sorted list of the points by their y coordinates. At each recursive step, we form a subset of the points in the region sorted by their y coordinates from the already sorted set of all points sorted by their y coordinates, which can be done with $O(n)$ comparisons.

Beginning with a point in the strip with the smallest y coordinate, we successively examine each point in the strip, computing the distance between this point and all other points in the strip that have larger y coordinates that could lie at a distance less than d from this point. Note that to examine a point p , we need only consider the distances between p and points in the set that lie within the rectangle of height d and width $2d$ with p on its base and with vertical sides at distance d from ℓ .

We can show that there are at most eight points from the set, including p , in or on this $2d \times d$ rectangle. To see this, note that there can be at most one point in each of the eight $d/2 \times d/2$ squares shown in Figure 2. This follows because the farthest apart points can be on or within one of these squares is the diagonal length $d/\sqrt{2}$ (which can be found using the Pythagorean theorem), which is less than d , and each of these $d/2 \times d/2$ squares lies entirely within the left region or the right region. This means that at this stage we need only compare at most seven distances, the distances between p and the seven or fewer other points in or on the rectangle, with d .

Because the total number of points in the strip of width $2d$ does not exceed n (the total number of points in the set), at most $7n$ distances need to be compared with d to find the minimum distance between points. That is, there are only $7n$ possible distances that could be less than d . Consequently, once the merge sort has been used to sort the pairs according to their x coordinates and according to their y coordinates, we find that the increasing function $f(n)$ satisfying the recurrence relation

$$f(n) = 2f(n/2) + 7n,$$

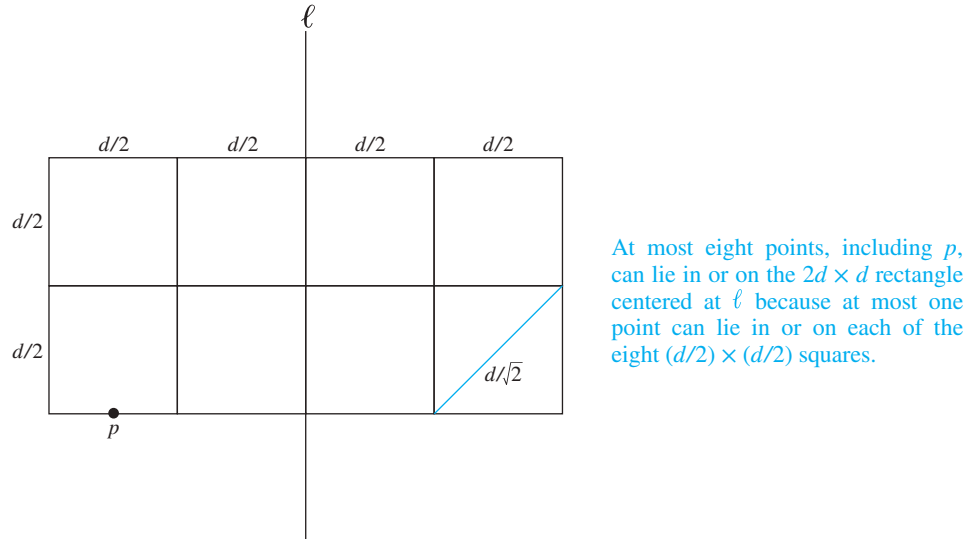


FIGURE 2 Showing that there are at most seven other points to consider for each point in the strip.

where $f(2) = 1$, exceeds the number of comparisons needed to solve the closest-pair problem for n points. By the master theorem (Theorem 2), it follows that $f(n)$ is $O(n \log n)$. The two sorts of points by their x coordinates and by their y coordinates each can be done using $O(n \log n)$ comparisons, by using the merge sort, and the sorted subsets of these coordinates at each of the $O(\log n)$ steps of the algorithm can be done using $O(n)$ comparisons each. Thus, we find that the closest-pair problem can be solved using $O(n \log n)$ comparisons. ◀

Exercises

- How many comparisons are needed for a binary search in a set of 64 elements?
- How many comparisons are needed to locate the maximum and minimum elements in a sequence with 128 elements using the algorithm in Example 2?
- Multiply $(1110)_2$ and $(1010)_2$ using the fast multiplication algorithm.
- Express the fast multiplication algorithm in pseudocode.
- Determine a value for the constant C in Example 4 and use it to estimate the number of bit operations needed to multiply two 64-bit integers using the fast multiplication algorithm.
- How many operations are needed to multiply two 32×32 matrices using the algorithm referred to in Example 5?
- Suppose that $f(n) = f(n/3) + 1$ when n is a positive integer divisible by 3, and $f(1) = 1$. Find
 - $f(3)$.
 - $f(27)$.
 - $f(729)$.
- Suppose that $f(n) = 2f(n/2) + 3$ when n is an even positive integer, and $f(1) = 5$. Find
 - $f(2)$.
 - $f(8)$.
 - $f(64)$.
 - $f(1024)$.
- Suppose that $f(n) = f(n/5) + 3n^2$ when n is a positive integer divisible by 5, and $f(1) = 4$. Find
 - $f(5)$.
 - $f(125)$.
 - $f(3125)$.
- Find $f(n)$ when $n = 2^k$, where f satisfies the recurrence relation $f(n) = f(n/2) + 1$ with $f(1) = 1$.
- Give a big- O estimate for the function f in Exercise 10 if f is an increasing function.
- Find $f(n)$ when $n = 3^k$, where f satisfies the recurrence relation $f(n) = 2f(n/3) + 4$ with $f(1) = 1$.
- Give a big- O estimate for the function f in Exercise 12 if f is an increasing function.
- Suppose that there are $n = 2^k$ teams in an elimination tournament, where there are $n/2$ games in the first round, with the $n/2 = 2^{k-1}$ winners playing in the second round, and so on. Develop a recurrence relation for the number of rounds in the tournament.
- How many rounds are in the elimination tournament described in Exercise 14 when there are 32 teams?
- Solve the recurrence relation for the number of rounds in the tournament described in Exercise 14.
- Suppose that the votes of n people for different candidates (where there can be more than two candidates) for a particular office are the elements of a sequence. A person wins the election if this person receives a majority of the votes.

- a) Devise a divide-and-conquer algorithm that determines whether a candidate received a majority and, if so, determine who this candidate is. [Hint: Assume that n is even and split the sequence of votes into two sequences, each with $n/2$ elements. Note that a candidate could not have received a majority of votes without receiving a majority of votes in at least one of the two halves.]
- b) Use the master theorem to give a big- O estimate for the number of comparisons needed by the algorithm you devised in part (a).
18. Suppose that each person in a group of n people votes for exactly two people from a slate of candidates to fill two positions on a committee. The top two finishers both win positions as long as each receives more than $n/2$ votes.
- a) Devise a divide-and-conquer algorithm that determines whether the two candidates who received the most votes each received at least $n/2$ votes and, if so, determine who these two candidates are.
- b) Use the master theorem to give a big- O estimate for the number of comparisons needed by the algorithm you devised in part (a).
19. a) Set up a divide-and-conquer recurrence relation for the number of multiplications required to compute x^n , where x is a real number and n is a positive integer, using the recursive algorithm from Exercise 26 in Section 5.4.
- b) Use the recurrence relation you found in part (a) to construct a big- O estimate for the number of multiplications used to compute x^n using the recursive algorithm.
20. a) Set up a divide-and-conquer recurrence relation for the number of modular multiplications required to compute $a^n \bmod m$, where a , m , and n are positive integers, using the recursive algorithms from Example 4 in Section 5.4.
- b) Use the recurrence relation you found in part (a) to construct a big- O estimate for the number of modular multiplications used to compute $a^n \bmod m$ using the recursive algorithm.
21. Suppose that the function f satisfies the recurrence relation $f(n) = 2f(\sqrt{n}) + 1$ whenever n is a perfect square greater than 1 and $f(2) = 1$.
- a) Find $f(16)$.
- b) Give a big- O estimate for $f(n)$. [Hint: Make the substitution $m = \log n$.]
22. Suppose that the function f satisfies the recurrence relation $f(n) = 2f(\sqrt{n}) + \log n$ whenever n is a perfect square greater than 1 and $f(2) = 1$.
- a) Find $f(16)$.
- b) Find a big- O estimate for $f(n)$. [Hint: Make the substitution $m = \log n$.]
- **23. This exercise deals with the problem of finding the largest sum of consecutive terms of a sequence of n real numbers. When all terms are positive, the sum of all terms provides the answer, but the situation is more complicated when some terms are negative. For example, the maximum sum of consecutive terms of the sequence $-2, 3, -1, 6, -7, 4$ is $3 + (-1) + 6 = 8$. (This exercise is based on [Be86].) Recall that in Exercise 56 in Section 8.1 we developed a dynamic programming algorithm for solving this problem. Here, we first look at the brute-force algorithm for solving this problem; then we develop a divide-and-conquer algorithm for solving it.
- a) Use pseudocode to describe an algorithm that solves this problem by finding the sums of consecutive terms starting with the first term, the sums of consecutive terms starting with the second term, and so on, keeping track of the maximum sum found so far as the algorithm proceeds.
- b) Determine the computational complexity of the algorithm in part (a) in terms of the number of sums computed and the number of comparisons made.
- c) Devise a divide-and-conquer algorithm to solve this problem. [Hint: Assume that there are an even number of terms in the sequence and split the sequence into two halves. Explain how to handle the case when the maximum sum of consecutive terms includes terms in both halves.]
- d) Use the algorithm from part (c) to find the maximum sum of consecutive terms of each of the sequences: $-2, 4, -1, 3, 5, -6, 1, 2$; $4, 1, -3, 7, -1, -5, 3, -2$; and $-1, 6, 3, -4, -5, 8, -1, 7$.
- e) Find a recurrence relation for the number of sums and comparisons used by the divide-and-conquer algorithm from part (c).
- f) Use the master theorem to estimate the computational complexity of the divide-and-conquer algorithm. How does it compare in terms of computational complexity with the algorithm from part (a)?
24. Apply the algorithm described in Example 12 for finding the closest pair of points, using the Euclidean distance between points, to find the closest pair of the points $(1, 3)$, $(1, 7)$, $(2, 4)$, $(2, 9)$, $(3, 1)$, $(3, 5)$, $(4, 3)$, and $(4, 7)$.
25. Apply the algorithm described in Example 12 for finding the closest pair of points, using the Euclidean distance between points, to find the closest pair of the points $(1, 2)$, $(1, 6)$, $(2, 4)$, $(2, 8)$, $(3, 1)$, $(3, 6)$, $(3, 10)$, $(4, 3)$, $(5, 1)$, $(5, 5)$, $(5, 9)$, $(6, 7)$, $(7, 1)$, $(7, 4)$, $(7, 9)$, and $(8, 6)$.
- *26. Use pseudocode to describe the recursive algorithm for solving the closest-pair problem as described in Example 12.
27. Construct a variation of the algorithm described in Example 12 along with justifications of the steps used by the algorithm to find the smallest distance between two points if the distance between two points is defined to be $d((x_i, y_i), (x_j, y_j)) = \max(|x_i - x_j|, |y_i - y_j|)$.
- *28. Suppose someone picks a number x from a set of n numbers. A second person tries to guess the number by successively selecting subsets of the n numbers and asking the first person whether x is in each set. The first person answers either “yes” or “no.” When the first

Links

person answers each query truthfully, we can find x using $\log n$ queries by successively splitting the sets used in each query in half. Ulam's problem, proposed by Stanislaw Ulam in 1976, asks for the number of queries required to find x , supposing that the first person is allowed to lie exactly once.

- Show that by asking each question twice, given a number x and a set with n elements, and asking one more question when we find the lie, Ulam's problem can be solved using $2 \log n + 1$ queries.
- Show that by dividing the initial set of n elements into four parts, each with $n/4$ elements, $1/4$ of the elements can be eliminated using two queries. [Hint: Use two queries, where each of the queries asks whether the element is in the union of two of the subsets with $n/4$ elements and where one of the subsets of $n/4$ elements is used in both queries.]
- Show from part (b) that if $f(n)$ equals the number of queries used to solve Ulam's problem using the method from part (b) and n is divisible by 4, then $f(n) = f(3n/4) + 2$.
- Solve the recurrence relation in part (c) for $f(n)$.
- Is the naive way to solve Ulam's problem by asking each question twice or the divide-and-conquer method based on part (b) more efficient? The most efficient way to solve Ulam's problem has been determined by A. Pelc [Pe87].

In Exercises 29–33, assume that f is an increasing function satisfying the recurrence relation $f(n) = af(n/b) + cn^d$, where $a \geq 1$, b is an integer greater than 1, and c and d are positive real numbers. These exercises supply a proof of Theorem 2.

- *29. Show that if $a = b^d$ and n is a power of b , then $f(n) = f(1)n^d + cn^d \log_b n$.
30. Use Exercise 29 to show that if $a = b^d$, then $f(n)$ is $O(n^d \log n)$.
- *31. Show that if $a \neq b^d$ and n is a power of b , then $f(n) = C_1 n^d + C_2 n^{\log_b a}$, where $C_1 = b^d c / (b^d - a)$ and $C_2 = f(1) + b^d c / (a - b^d)$.
32. Use Exercise 31 to show that if $a < b^d$, then $f(n)$ is $O(n^d)$.
33. Use Exercise 31 to show that if $a > b^d$, then $f(n)$ is $O(n^{\log_b a})$.
34. Find $f(n)$ when $n = 4^k$, where f satisfies the recurrence relation $f(n) = 5f(n/4) + 6n$, with $f(1) = 1$.
35. Give a big- O estimate for the function f in Exercise 34 if f is an increasing function.
36. Find $f(n)$ when $n = 2^k$, where f satisfies the recurrence relation $f(n) = 8f(n/2) + n^2$ with $f(1) = 1$.
37. Give a big- O estimate for the function f in Exercise 36 if f is an increasing function.

8.4 Generating Functions

8.4.1 Introduction

Links

Generating functions are used to represent sequences efficiently by coding the terms of a sequence as coefficients of powers of a variable x in a formal power series. Generating functions can be used to solve many types of counting problems, such as the number of ways to select or distribute objects of different kinds, subject to a variety of constraints, and the number of ways to make change for a dollar using coins of different denominations. Generating functions can be used to solve recurrence relations by translating a recurrence relation for the terms of a sequence into an equation involving a generating function. This equation can then be solved to find a closed form for the generating function. From this closed form, the coefficients of the power series for the generating function can be found, solving the original recurrence relation. Generating functions can also be used to prove combinatorial identities by taking advantage of relatively simple relationships between functions that can be translated into identities involving the terms of sequences. Generating functions are a helpful tool for studying many properties of sequences besides those described in this section, such as their use for establishing asymptotic formulae for the terms of a sequence.

We begin with the definition of the generating function for a sequence.

Definition 1

The *generating function for the sequence* $a_0, a_1, \dots, a_k, \dots$ of real numbers is the infinite series

$$G(x) = a_0 + a_1x + \dots + a_kx^k + \dots = \sum_{k=0}^{\infty} a_kx^k.$$

Remark: The generating function for $\{a_k\}$ given in Definition 1 is sometimes called the **ordinary generating function** of $\{a_k\}$ to distinguish it from other types of generating functions for this sequence.

EXAMPLE 1 The generating functions for the sequences $\{a_k\}$ with $a_k = 3$, $a_k = k + 1$, and $a_k = 2^k$ are $\sum_{k=0}^{\infty} 3x^k$, $\sum_{k=0}^{\infty} (k + 1)x^k$, and $\sum_{k=0}^{\infty} 2^k x^k$, respectively. 

Extra Examples 

We can define generating functions for finite sequences of real numbers by extending a finite sequence $a_0, a_1, \dots, a_n$ into an infinite sequence by setting $a_{n+1} = 0$, $a_{n+2} = 0$, and so on. The generating function $G(x)$ of this infinite sequence $\{a_n\}$ is a polynomial of degree n because no terms of the form $a_j x^j$ with $j > n$ occur, that is,

$$G(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

EXAMPLE 2 What is the generating function for the sequence 1, 1, 1, 1, 1, 1?

Solution: The generating function of 1, 1, 1, 1, 1, 1 is

$$1 + x + x^2 + x^3 + x^4 + x^5.$$

By Theorem 1 of Section 2.4 we have

$$(x^6 - 1)/(x - 1) = 1 + x + x^2 + x^3 + x^4 + x^5$$

when $x \neq 1$. Consequently, $G(x) = (x^6 - 1)/(x - 1)$ is the generating function of the sequence 1, 1, 1, 1, 1, 1. [Because the powers of x are only place holders for the terms of the sequence in a generating function, we do not need to worry that $G(1)$ is undefined.] 

EXAMPLE 3 Let m be a positive integer. Let $a_k = C(m, k)$, for $k = 0, 1, 2, \dots, m$. What is the generating function for the sequence $a_0, a_1, \dots, a_m$?

Solution: The generating function for this sequence is

$$G(x) = C(m, 0) + C(m, 1)x + C(m, 2)x^2 + \cdots + C(m, m)x^m.$$

The binomial theorem shows that $G(x) = (1 + x)^m$. 

8.4.2 Useful Facts About Power Series

When generating functions are used to solve counting problems, they are usually considered to be **formal power series**. As such, they are treated as algebraic objects; questions about their convergence are ignored. However, when formal power series are convergent, valid operations carry over to their use as formal power series. We will take advantage of the power series of particular functions around $x = 0$. These power series are unique and have a positive radius of convergence. Readers familiar with calculus can consult textbooks on this subject for details about power series, including the convergence of the series we use here.

We will now state some widely important facts about infinite series used when working with generating functions. These facts can be found in calculus texts.

EXAMPLE 4 The function $f(x) = 1/(1 - x)$ is the generating function of the sequence 1, 1, 1, 1, ..., because

$$1/(1 - x) = 1 + x + x^2 + \dots$$

for $|x| < 1$. 

EXAMPLE 5 The function $f(x) = 1/(1 - ax)$ is the generating function of the sequence 1, a , a^2 , a^3 , ..., because

$$1/(1 - ax) = 1 + ax + a^2x^2 + \dots$$

when $|ax| < 1$, or equivalently, for $|x| < 1/|a|$ for $a \neq 0$. 

We also will need some results on how to add and how to multiply two generating functions. Proofs of these results can be found in calculus texts.

THEOREM 1

Let $f(x) = \sum_{k=0}^{\infty} a_k x^k$ and $g(x) = \sum_{k=0}^{\infty} b_k x^k$. Then

$$f(x) + g(x) = \sum_{k=0}^{\infty} (a_k + b_k) x^k \quad \text{and} \quad f(x)g(x) = \sum_{k=0}^{\infty} \left(\sum_{j=0}^k a_j b_{k-j} \right) x^k.$$

Remark: Theorem 1 is valid only for power series that converge in an interval, as all series considered in this section do. However, the theory of generating functions is not limited to such series. In the case of series that do not converge, the statements in Theorem 1 can be taken as definitions of addition and multiplication of generating functions.

We will illustrate how Theorem 1 can be used with Example 6.

EXAMPLE 6 Let $f(x) = 1/(1 - x)^2$. Use Example 4 to find the coefficients $a_0, a_1, a_2, \dots$ in the expansion $f(x) = \sum_{k=0}^{\infty} a_k x^k$.

Solution: From Example 4 we see that

$$1/(1 - x) = 1 + x + x^2 + x^3 + \dots$$

Hence, from Theorem 1, we have

$$1/(1 - x)^2 = \sum_{k=0}^{\infty} \left(\sum_{j=0}^k 1 \right) x^k = \sum_{k=0}^{\infty} (k + 1) x^k.$$


Remark: This result also can be derived from Example 4 by differentiation. Taking derivatives is a useful technique for producing new identities from existing identities for generating functions.

To use generating functions to solve many important counting problems, we will need to apply the binomial theorem for exponents that are not positive integers. Before we state an extended version of the binomial theorem, we need to define extended binomial coefficients.

Definition 2

Let u be a real number and k a nonnegative integer. Then the *extended binomial coefficient* $\binom{u}{k}$ is defined by

$$\binom{u}{k} = \begin{cases} u(u-1) \cdots (u-k+1)/k! & \text{if } k > 0, \\ 1 & \text{if } k = 0. \end{cases}$$

EXAMPLE 7 Find the values of the extended binomial coefficients $\binom{-2}{3}$ and $\binom{1/2}{3}$.

Solution: Taking $u = -2$ and $k = 3$ in Definition 2 gives us

$$\binom{-2}{3} = \frac{(-2)(-3)(-4)}{3!} = -4.$$

Similarly, taking $u = 1/2$ and $k = 3$ gives us

$$\begin{aligned} \binom{1/2}{3} &= \frac{(1/2)(1/2-1)(1/2-2)}{3!} \\ &= (1/2)(-1/2)(-3/2)/6 \\ &= 1/16. \end{aligned}$$

Example 8 provides a useful formula for extended binomial coefficients when the top parameter is a negative integer. It will be useful in our subsequent discussions.

EXAMPLE 8 When the top parameter is a negative integer, the extended binomial coefficient can be expressed in terms of an ordinary binomial coefficient. To see that this is the case, note that

$$\begin{aligned} \binom{-n}{r} &= \frac{(-n)(-n-1) \cdots (-n-r+1)}{r!} && \text{by definition of extended binomial coefficient} \\ &= \frac{(-1)^r n(n+1) \cdots (n+r-1)}{r!} && \text{factoring out } -1 \text{ from each term in the numerator} \\ &= \frac{(-1)^r (n+r-1)(n+r-2) \cdots n}{r!} && \text{by the commutative law for multiplication} \\ &= \frac{(-1)^r (n+r-1)!}{r!(n-1)!} && \begin{array}{l} \text{multiplying both the numerator and denominator} \\ \text{by } (n-1)! \end{array} \\ &= (-1)^r \binom{n+r-1}{r} && \text{by the definition of binomial coefficients} \\ &= (-1)^r C(n+r-1, r) && \begin{array}{l} \text{using alternative notation for binomial} \\ \text{coefficients.} \end{array} \end{aligned}$$

We now state the extended binomial theorem.

THEOREM 2 THE EXTENDED BINOMIAL THEOREM Let x be a real number with $|x| < 1$ and let u be a real number. Then

$$(1+x)^u = \sum_{k=0}^{\infty} \binom{u}{k} x^k.$$

Theorem 2 can be proved using the theory of Maclaurin series. We leave its proof to the reader with a familiarity with this part of calculus.

Remark: When u is a positive integer, the extended binomial theorem reduces to the binomial theorem presented in Section 6.4, because in that case $\binom{u}{k} = 0$ if $k > u$.

Example 9 illustrates the use of Theorem 2 when the exponent is a negative integer.

EXAMPLE 9 Find the generating functions for $(1+x)^{-n}$ and $(1-x)^{-n}$, where n is a positive integer, using the extended binomial theorem.

Solution: By the extended binomial theorem, it follows that

$$(1+x)^{-n} = \sum_{k=0}^{\infty} \binom{-n}{k} x^k.$$

Using Example 8, which provides a simple formula for $\binom{-n}{k}$, we obtain

$$(1+x)^{-n} = \sum_{k=0}^{\infty} (-1)^k C(n+k-1, k) x^k.$$

Replacing x by $-x$, we find that

$$(1-x)^{-n} = \sum_{k=0}^{\infty} C(n+k-1, k) x^k.$$

Table 1 presents a useful summary of some generating functions that arise frequently.

Remark: Note that the second and third formulae in this table can be deduced from the first formula by substituting ax and x^r for x , respectively. Similarly, the sixth and seventh formulae can be deduced from the fifth formula using the same substitutions. The tenth and eleventh can be deduced from the ninth formula by substituting $-x$ and ax for x , respectively. Also, some of the formulae in this table can be derived from other formulae using methods from calculus (such as differentiation and integration). Students are encouraged to know the core formulae in this table (that is, formulae from which the others can be derived, perhaps the first, fourth, fifth, eighth, ninth, twelfth, and thirteenth formulae) and understand how to derive the other formulae from these core formulae.

TABLE 1 Useful Generating Functions.

$G(x)$	a_k
$(1+x)^n = \sum_{k=0}^n C(n, k)x^k$ $= 1 + C(n, 1)x + C(n, 2)x^2 + \cdots + x^n$	$C(n, k)$
$(1+ax)^n = \sum_{k=0}^n C(n, k)a^k x^k$ $= 1 + C(n, 1)ax + C(n, 2)a^2x^2 + \cdots + a^n x^n$	$C(n, k)a^k$
$(1+x^r)^n = \sum_{k=0}^n C(n, k)x^{rk}$ $= 1 + C(n, 1)x^r + C(n, 2)x^{2r} + \cdots + x^{rn}$	$C(n, k/r)$ if $r \mid k$; 0 otherwise
$\frac{1-x^{n+1}}{1-x} = \sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n$	1 if $k \leq n$; 0 otherwise
$\frac{1}{1-x} = \sum_{k=0}^{\infty} x^k = 1 + x + x^2 + \cdots$	1
$\frac{1}{1-ax} = \sum_{k=0}^{\infty} a^k x^k = 1 + ax + a^2x^2 + \cdots$	a^k
$\frac{1}{1-x^r} = \sum_{k=0}^{\infty} x^{rk} = 1 + x^r + x^{2r} + \cdots$	1 if $r \mid k$; 0 otherwise
$\frac{1}{(1-x)^2} = \sum_{k=0}^{\infty} (k+1)x^k = 1 + 2x + 3x^2 + \cdots$	$k+1$
$\frac{1}{(1-x)^n} = \sum_{k=0}^{\infty} C(n+k-1, k)x^k$ $= 1 + C(n, 1)x + C(n+1, 2)x^2 + \cdots$	$C(n+k-1, k) = C(n+k-1, n-1)$
$\frac{1}{(1+x)^n} = \sum_{k=0}^{\infty} C(n+k-1, k)(-1)^k x^k$ $= 1 - C(n, 1)x + C(n+1, 2)x^2 - \cdots$	$(-1)^k C(n+k-1, k) = (-1)^k C(n+k-1, n-1)$
$\frac{1}{(1-ax)^n} = \sum_{k=0}^{\infty} C(n+k-1, k)a^k x^k$ $= 1 + C(n, 1)ax + C(n+1, 2)a^2x^2 + \cdots$	$C(n+k-1, k)a^k = C(n+k-1, n-1)a^k$
$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$	$1/k!$
$\ln(1+x) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} x^k = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$	$(-1)^{k+1}/k$

Note: The series for the last two generating functions can be found in most calculus books when power series are discussed.

8.4.3 Counting Problems and Generating Functions

Generating functions can be used to solve a wide variety of counting problems. In particular, they can be used to count the number of combinations of various types. In Chapter 6 we developed techniques to count the r -combinations from a set with n elements when repetition is allowed and additional constraints may exist. Such problems are equivalent to counting the solutions to equations of the form

$$e_1 + e_2 + \cdots + e_n = C,$$

where C is a constant and each e_i is a nonnegative integer that may be subject to a specified constraint. Generating functions can also be used to solve counting problems of this type, as Examples 10–12 show.

EXAMPLE 10 Find the number of solutions of

*Extra
Examples* 

$$e_1 + e_2 + e_3 = 17,$$

where e_1, e_2 , and e_3 are nonnegative integers with $2 \leq e_1 \leq 5$, $3 \leq e_2 \leq 6$, and $4 \leq e_3 \leq 7$.

Solution: The number of solutions with the indicated constraints is the coefficient of x^{17} in the expansion of

$$(x^2 + x^3 + x^4 + x^5)(x^3 + x^4 + x^5 + x^6)(x^4 + x^5 + x^6 + x^7).$$

This follows because we obtain a term equal to x^{17} in the product by picking a term in the first sum x^{e_1} , a term in the second sum x^{e_2} , and a term in the third sum x^{e_3} , where the exponents e_1, e_2 , and e_3 satisfy the equation $e_1 + e_2 + e_3 = 17$ and the given constraints.

It is not hard to see that the coefficient of x^{17} in this product is 3. Hence, there are three solutions. (Note that the calculating of this coefficient involves about as much work as enumerating all the solutions of the equation with the given constraints. However, the method that this illustrates often can be used to solve wide classes of counting problems with special formulae, as we will see. Furthermore, a computer algebra system can be used to do such computations.) 

EXAMPLE 11 In how many different ways can eight identical cookies be distributed among three distinct children if each child receives at least two cookies and no more than four cookies?

Solution: Because each child receives at least two but no more than four cookies, for each child there is a factor equal to

$$(x^2 + x^3 + x^4)$$

in the generating function for the sequence $\{c_n\}$, where c_n is the number of ways to distribute n cookies. Because there are three children, this generating function is

$$(x^2 + x^3 + x^4)^3.$$

We need the coefficient of x^8 in this product. The reason is that the x^8 terms in the expansion correspond to the ways that three terms can be selected, with one from each factor, that have exponents adding up to 8. Furthermore, the exponents of the term from the first, second, and third factors are the numbers of cookies the first, second, and third children receive, respectively. Computation shows that this coefficient equals 6. Hence, there are six ways to distribute the cookies so that each child receives at least two, but no more than four, cookies. 

EXAMPLE 12 Use generating functions to determine the number of ways to insert tokens worth \$1, \$2, and \$5 into a vending machine to pay for an item that costs r dollars in both the cases when the order in which the tokens are inserted does not matter and when the order does matter. (For example, there are two ways to pay for an item that costs \$3 when the order in which the tokens are inserted does not matter: inserting three \$1 tokens or one \$1 token and a \$2 token. When the order matters, there are three ways: inserting three \$1 tokens, inserting a \$1 token and then a \$2 token, or inserting a \$2 token and then a \$1 token.)

Solution: Consider the case when the order in which the tokens are inserted does not matter. Here, all we care about is the number of each token used to produce a total of r dollars. Because we can use any number of \$1 tokens, any number of \$2 tokens, and any number of \$5 tokens, the answer is the coefficient of x^r in the generating function

$$(1 + x + x^2 + x^3 + \cdots)(1 + x^2 + x^4 + x^6 + \cdots)(1 + x^5 + x^{10} + x^{15} + \cdots).$$

(The first factor in this product represents the \$1 tokens used, the second the \$2 tokens used, and the third the \$5 tokens used.) For example, the number of ways to pay for an item costing \$7 using \$1, \$2, and \$5 tokens is given by the coefficient of x^7 in this expansion, which equals 6.

When the order in which the tokens are inserted matters, the number of ways to insert exactly n tokens to produce a total of r dollars is the coefficient of x^r in

$$(x + x^2 + x^5)^n,$$

because each of the r tokens may be a \$1 token, a \$2 token, or a \$5 token. Because any number of tokens may be inserted, the number of ways to produce r dollars using \$1, \$2, or \$5 tokens, when the order in which the tokens are inserted matters, is the coefficient of x^r in

$$\begin{aligned} 1 + (x + x^2 + x^5) + (x + x^2 + x^5)^2 + \cdots &= \frac{1}{1 - (x + x^2 + x^5)} \\ &= \frac{1}{1 - x - x^2 - x^5}, \end{aligned}$$

where we have added the number of ways to insert 0 tokens, 1 token, 2 tokens, 3 tokens, and so on, and where we have used the identity $1/(1 - x) = 1 + x + x^2 + \cdots$ with x replaced with $x + x^2 + x^5$. For example, the number of ways to pay for an item costing \$7 using \$1, \$2, and \$5 tokens, when the order in which the tokens are used matters, is the coefficient of x^7 in this expansion, which equals 26. [Hint: To see that this coefficient equals 26 requires the addition of the coefficients of x^7 in the expansions $(x + x^2 + x^5)^k$ for $2 \leq k \leq 7$. This can be done by hand with considerable computation, or a computer algebra system can be used.] ◀

Example 13 shows the versatility of generating functions when used to solve problems with differing assumptions.

EXAMPLE 13 Use generating functions to find the number of k -combinations of a set with n elements. Assume that the binomial theorem has already been established.

Solution: Each of the n elements in the set contributes the term $(1 + x)$ to the generating function $f(x) = \sum_{k=0}^n a_k x^k$. Here $f(x)$ is the generating function for $\{a_k\}$, where a_k represents the number of k -combinations of a set with n elements. Hence,

$$f(x) = (1 + x)^n.$$

But by the binomial theorem, we have

$$f(x) = \sum_{k=0}^n \binom{n}{k} x^k,$$

where

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Hence, $C(n, k)$, the number of k -combinations of a set with n elements, is

$$\frac{n!}{k!(n-k)!}.$$

Remark: We proved the binomial theorem in Section 6.4 using the formula for the number of r -combinations of a set with n elements. This example shows that the binomial theorem, which can be proved by mathematical induction, can be used to derive the formula for the number of r -combinations of a set with n elements.

EXAMPLE 14 Use generating functions to find the number of r -combinations from a set with n elements when repetition of elements is allowed.

Solution: Let $G(x)$ be the generating function for the sequence $\{a_r\}$, where a_r equals the number of r -combinations of a set with n elements with repetitions allowed. That is, $G(x) = \sum_{r=0}^{\infty} a_r x^r$. Because we can select any number of a particular member of the set with n elements when we form an r -combination with repetition allowed, each of the n elements contributes $(1 + x + x^2 + x^3 + \cdots)$ to a product expansion for $G(x)$. Each element contributes this factor because it may be selected zero times, one time, two times, three times, and so on, when an r -combination is formed (with a total of r elements selected). Because there are n elements in the set and each contributes this same factor to $G(x)$, we have

$$G(x) = (1 + x + x^2 + \cdots)^n.$$

As long as $|x| < 1$, we have $1 + x + x^2 + \cdots = 1/(1 - x)$, so

$$G(x) = 1/(1 - x)^n = (1 - x)^{-n}.$$

Applying the extended binomial theorem (Theorem 2), it follows that

$$(1 - x)^{-n} = (1 + (-x))^{-n} = \sum_{r=0}^{\infty} \binom{-n}{r} (-x)^r.$$

The number of r -combinations of a set with n elements with repetitions allowed, when r is a positive integer, is the coefficient a_r of x^r in this sum. Consequently, using Example 8 we find that a_r equals

$$\begin{aligned} \binom{-n}{r} (-1)^r &= (-1)^r C(n + r - 1, r) \cdot (-1)^r \\ &= C(n + r - 1, r). \end{aligned}$$

Note that the result in Example 14 is the same result we stated as Theorem 2 in Section 6.5.

EXAMPLE 15 Use generating functions to find the number of ways to select r objects of n different kinds if we must select at least one object of each kind.

Solution: Because we need to select at least one object of each kind, each of the n kinds of objects contributes the factor $(x + x^2 + x^3 + \cdots)$ to the generating function $G(x)$ for the sequence $\{a_r\}$, where a_r is the number of ways to select r objects of n different kinds if we need at least one object of each kind. Hence,

$$G(x) = (x + x^2 + x^3 + \cdots)^n = x^n(1 + x + x^2 + \cdots)^n = x^n/(1 - x)^n.$$

Using the extended binomial theorem and Example 8, we have

$$\begin{aligned} G(x) &= x^n/(1 - x)^n \\ &= x^n \cdot (1 - x)^{-n} \\ &= x^n \sum_{r=0}^{\infty} \binom{-n}{r} (-x)^r \\ &= x^n \sum_{r=0}^{\infty} (-1)^r C(n + r - 1, r) (-1)^r x^r \\ &= \sum_{r=0}^{\infty} C(n + r - 1, r) x^{n+r} \\ &= \sum_{t=n}^{\infty} C(t - 1, t - n) x^t \\ &= \sum_{r=n}^{\infty} C(r - 1, r - n) x^r. \end{aligned}$$

We have shifted the summation in the next-to-last equality by setting $t = n + r$ so that $t = n$ when $r = 0$ and $n + r - 1 = t - 1$, and then we replaced t by r as the index of summation in the last equality to return to our original notation. Hence, there are $C(r - 1, r - n)$ ways to select r objects of n different kinds if we must select at least one object of each kind. 

8.4.4 Using Generating Functions to Solve Recurrence Relations

We can find the solution to a recurrence relation and its initial conditions by finding an explicit formula for the associated generating function. This is illustrated in Examples 16 and 17.

EXAMPLE 16 Solve the recurrence relation $a_k = 3a_{k-1}$ for $k = 1, 2, 3, \dots$ and initial condition $a_0 = 2$.

Extra Examples 

Solution: Let $G(x)$ be the generating function for the sequence $\{a_k\}$, that is, $G(x) = \sum_{k=0}^{\infty} a_k x^k$. First note that

$$xG(x) = \sum_{k=0}^{\infty} a_k x^{k+1} = \sum_{k=1}^{\infty} a_{k-1} x^k.$$

Using the recurrence relation, we see that

$$\begin{aligned} G(x) - 3xG(x) &= \sum_{k=0}^{\infty} a_k x^k - 3 \sum_{k=1}^{\infty} a_{k-1} x^k \\ &= a_0 + \sum_{k=1}^{\infty} (a_k - 3a_{k-1}) x^k \\ &= 2, \end{aligned}$$

because $a_0 = 2$ and $a_k = 3a_{k-1}$. Thus,

$$G(x) - 3xG(x) = (1 - 3x)G(x) = 2.$$

Solving for $G(x)$ shows that $G(x) = 2/(1 - 3x)$. Using the identity $1/(1 - ax) = \sum_{k=0}^{\infty} a^k x^k$, from Table 1, we have

$$G(x) = 2 \sum_{k=0}^{\infty} 3^k x^k = \sum_{k=0}^{\infty} 2 \cdot 3^k x^k.$$

Consequently, $a_k = 2 \cdot 3^k$. 

EXAMPLE 17 Suppose that a valid codeword is an n -digit number in decimal notation containing an even number of 0s. Let a_n denote the number of valid codewords of length n . In Example 4 of Section 8.1 we showed that the sequence $\{a_n\}$ satisfies the recurrence relation

$$a_n = 8a_{n-1} + 10^{n-1}$$

and the initial condition $a_1 = 9$. Use generating functions to find an explicit formula for a_n .

Solution: To make our work with generating functions simpler, we extend this sequence by setting $a_0 = 1$; when we assign this value to a_0 and use the recurrence relation, we have $a_1 = 8a_0 + 10^0 = 8 + 1 = 9$, which is consistent with our original initial condition. (It also makes sense because there is one code word of length 0—the empty string.)

We multiply both sides of the recurrence relation by x^n to obtain

$$a_n x^n = 8a_{n-1} x^n + 10^{n-1} x^n.$$

Let $G(x) = \sum_{n=0}^{\infty} a_n x^n$ be the generating function of the sequence $a_0, a_1, a_2, \dots$. We sum both sides of the last equation starting with $n = 1$, to find that

$$\begin{aligned} G(x) - 1 &= \sum_{n=1}^{\infty} a_n x^n = \sum_{n=1}^{\infty} (8a_{n-1} x^n + 10^{n-1} x^n) \\ &= 8 \sum_{n=1}^{\infty} a_{n-1} x^n + \sum_{n=1}^{\infty} 10^{n-1} x^n \\ &= 8x \sum_{n=1}^{\infty} a_{n-1} x^{n-1} + x \sum_{n=1}^{\infty} 10^{n-1} x^{n-1} \\ &= 8x \sum_{n=0}^{\infty} a_n x^n + x \sum_{n=0}^{\infty} 10^n x^n \\ &= 8xG(x) + x/(1 - 10x), \end{aligned}$$

where we have used Example 5 to evaluate the second summation. Therefore, we have

$$G(x) - 1 = 8xG(x) + x/(1 - 10x).$$

Solving for $G(x)$ shows that

$$G(x) = \frac{1 - 9x}{(1 - 8x)(1 - 10x)}.$$

Expanding the right-hand side of this equation into partial fractions (as is done in the integration of rational functions studied in calculus) gives

$$G(x) = \frac{1}{2} \left(\frac{1}{1 - 8x} + \frac{1}{1 - 10x} \right).$$

Using Example 5 twice (once with $a = 8$ and once with $a = 10$) gives

$$\begin{aligned} G(x) &= \frac{1}{2} \left(\sum_{n=0}^{\infty} 8^n x^n + \sum_{n=0}^{\infty} 10^n x^n \right) \\ &= \sum_{n=0}^{\infty} \frac{1}{2} (8^n + 10^n) x^n. \end{aligned}$$

Consequently, we have shown that

$$a_n = \frac{1}{2} (8^n + 10^n).$$



8.4.5 Proving Identities via Generating Functions

In Chapter 6 we saw how combinatorial identities could be established using combinatorial proofs. Here we will show that such identities, as well as identities for extended binomial coefficients, can be proved using generating functions. Sometimes the generating function approach is simpler than other approaches, especially when it is simpler to work with the closed form of a generating function than with the terms of the sequence themselves. We illustrate how generating functions can be used to prove identities with Example 18.

EXAMPLE 18 Use generating functions to show that

$$\sum_{k=0}^n C(n, k)^2 = C(2n, n)$$

whenever n is a positive integer.

Solution: First note that by the binomial theorem $C(2n, n)$ is the coefficient of x^n in $(1 + x)^{2n}$. However, we also have

$$\begin{aligned} (1 + x)^{2n} &= [(1 + x)^n]^2 \\ &= [C(n, 0) + C(n, 1)x + C(n, 2)x^2 + \cdots + C(n, n)x^n]^2. \end{aligned}$$